

Zusammenfassung

1.1. THE 4 INGREDIENTS OF MACHINE-LEARNING

- ## 1.2. HOW TO REPRESENT WORDS IN A COMPUTER

1.2.1. One-hot representation

[illegible]

1.2.1.1. Disadvantages

- ### 1.2.2. Indexing

cat	0
mat	1
on	2
sat	3
the	4
.	5

“The cat sat on the mat.” $\rightarrow [4, 0, 3, 2, 4, 1, 5]$

This is the “*dense equivalent*” of one-hot encoding. Indexing is (often) used as a preprocessing step. These are more useful representations.

1.2.3. Distributed representation

A word can be *“defined” by context*. Words with *similar semantics share some context* (e.g., rat and cat are both animals).

Distributed representations can be **learned**. Example below: Similar traits are coloured similarly / at the same positions. We can subtract the properties of "Man" from "King" and add "Woman" to get the word "Queen".

King																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																	</
------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	----

Word to Vector

A **mathematical function maps** a **word** or the corresponding index (input) to a **high dimensional vector** (output). Machine-learning algorithms can **learn** this function. In neural networks this function is implemented in a **embedding layer**.

Advantages of vectors

Ideally, a “good” embedding **maps similar/related words to similar regions of the vector-space**. That is, nearby words have a semantic similarity. Once we have “good” vectors, we can do math. **The dot-product** (Skalarprodukt) is a measure of **similarity**. Vectors also can be added/subtracted.



For specific tasks, it is sufficient to have a notion of similarity between words. The question “how to understand words” becomes **“how to calculate similarity between words”**.

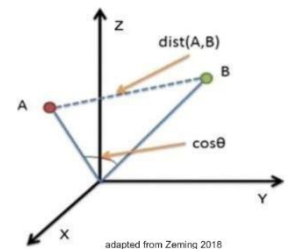
Basic properties of the dot product:

- The dot product between two vectors is **maximal** when they are both the same (going in the **same direction**). If the two vectors both have **norm / length 1**, then the **maximal** value is **1**.
- The dot product between two vectors is **zero** iff (if and only if) they share no components (are **perpendicular** / 90° difference / orthogonality)
- The dot product between two vectors is **minimal** (negative) when they are **anti-parallel** (pointing in opposite directions). If the two vectors both have **norm 1**, then this minimal value is **-1**.

Cosine-similarity or cosine-distance

A and B are vectors. Say A represents “cat”, B represents “dog”. The cosine-distance is a way to calculate how similar two words (vectors) are.

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} = \frac{\text{dot product of the vectors}}{\text{Length of the vectors multiplied}}$$



Calculation of cosine similarity

The **dot product** can be used to multiply two vectors of equal size. The result is a real number.

$$\begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} * \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} = a_1 * b_1 + a_2 * b_2 + a_3 * b_3$$

The **length / magnitude / norm** of a vector is calculated by squaring all the values of the vector, adding them together and then take the square root of the sum. The result is a real number.

$$\begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} = \sqrt{(a_1)^2 + (a_2)^2 + (a_3)^2}$$

Cosine-similarity:

$$\text{similarity}(A, B) = \frac{a_1 * b_1 + a_2 * b_2 + a_3 * b_3}{\sqrt{(a_1)^2 + (a_2)^2 + (a_3)^2} * \sqrt{(b_1)^2 + (b_2)^2 + (b_3)^2}}$$

The resulting similarity ranges from **-1** meaning **exactly opposite**, to **1** meaning **exactly the same** vector/word, with **0** indicating **orthogonality (perpendicular/ 90°)**.

There are two approaches to training a general language model: **Download and use a predefined language model** or the **usage and optimization of an Embedding Layer**. A known architecture for training embeddings is known under the term **word2vec**.

1.3. LARGE LANGUAGE MODELS (LLMs)

Large Language Models dominate the current success stories. LLMs are **large Artificial Neural Networks** that are used for translation, chat, Q&A, programming etc.

Current LLMs are **Transformer based**. Transformers are computational units with a particular structure and a trainable “Attention mechanism”. *Not further covered in AiFo*

Generative Language models produce sequences by **calculating a probability distribution over the next word given the past text**. They sample one word (or token) and repeat the process.

2. STATISTICS

2.1. RANDOM VARIABLES

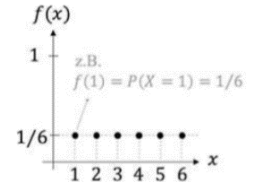
A random variable X is a variable that **takes a numerical value** x which **depends on a random experiment**. This is a way to represent outcomes of a random experiment as numbers.



- **Discrete:** X takes any of a finite set of values, e.g. $\{-8, 1.5, 2.693, 5\}$
- **Continuous:** X takes any value of an uncountable range, e.g. the real numbers in the interval $(2; 7)$.

2.1.1. Probability Mass Function (Wahrscheinlichkeitsfunktion)

Function $f(x)$ that provides the probability for each value x of a discrete random variable X . The tables below are PMFs, corresponds to graph. From the PDF we can calculate the expected value $E(X)$.



2.1.2. Example: Rolling a single dice

The discrete random variable X is the number observed when rolling a fair dice. The possible values and the probabilities they take are:

Value x of the random variable X	1	2	3	4	5	6
$\Pr(X = x)$ can also be written as $P(x)$, $p(x)$ or $P_x(x)$ (Probability that the random variable X takes the value x)	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$

2.1.3. Example 2: Sum of two dice

The discrete random variable X is the sum of eyes of two dice. ($6^2 = 36$ Options, only 2 dice combinations can result in a 3 (1/2 and 2/1), while for a 9, there are 4 combinations (6/3, 5/4, 4/5 and 3/6)

Value x	2	3	4	5	6	7	8	9	10	11	12
$\Pr(X = x)$	$\frac{1}{36}$	$\frac{2}{36}$	$\frac{3}{36}$	$\frac{4}{36}$	$\frac{5}{36}$	$\frac{6}{36}$	$\frac{5}{36}$	$\frac{4}{36}$	$\frac{3}{36}$	$\frac{2}{36}$	$\frac{1}{36}$

Probability that the sum is between 4 and 7 (sum of probabilities):

$$\Pr(4 \leq \text{sum} \leq 7) = \Pr(X = 4) + \Pr(X = 5) + \Pr(X = 6) + \Pr(X = 7) = \frac{3}{36} + \frac{4}{36} + \frac{5}{36} + \frac{6}{36} = \frac{18}{36} = 50\%$$

In Example 2, we have two random variables. X is the number of eyes on dice #1 and Y is the number of eyes of dice #2.

2.2. JOINT PROBABILITIES

The joint properties of two random variables are defined by the **Joint Probabilities Mass Function**.

2.2.1. Joint Probabilities with independent random variables

For independent random variables, the joint probability is the **product of the individual probabilities**. This is also true with more than two independent random variables.

Example: The **first** dice does **not affect** the probability of **the second** dice, so the probability for die #1 showing 5 and die #2 showing 4 is still $\frac{1}{36}$:

$$\Pr(X, Y) = \Pr(X) * \Pr(Y) \rightarrow \Pr(X = 5, Y = 4) = \Pr(X = 5) * \Pr(Y = 4) = \frac{1}{6} * \frac{1}{6} = \frac{1}{36}$$

2.2.2. Joint Probabilities with dependent random variables

If the **events are not independent**, the variables are dependent or correlated. Example:

- X : The event to observe clouds (0 = no clouds, 1 = small clouds, 2 = big clouds)
- Y : The event to observe that it rains (0 = no rain, 1 = light rain, 2 = moderate rain, 3 = heavy rain)

Given there are **small clouds**, what is the probability for **moderate rain**? This value cannot be read directly from the table, because all the probabilities in the full table together are 1, in this case however we only look at Y given that $X = 1$.

	$X = 0$	$X = 1$	$X = 2$
$Y = 0$	0.35	0.21	0.03
$Y = 1$	0.10	0.07	0.04
$Y = 2$	0.00	0.05	0.05
$Y = 3$	0.00	0.02	0.08

2.2.3. Marginal Probability

The probability of an event occurring, **irrespective of the outcome of another** random variable. For example, the probability of $Y = 2$ for all outcomes of X . If the two variables are visible in a table, then the marginal probability of one variable Y would be the **sum of probabilities** for the other variable X on the margin of the table. This is often used to "normalize" the values across a "row" or "column".

In other words, the probability of Y regardless of X is the sum of all probabilities of X where Y appears.

$$\text{Written as } \Pr(X) = \sum_Y \Pr(X, Y) \text{ or } \Pr(Y) = \sum_X \Pr(X, Y)$$

Example: The probability of rain, regardless of cloud size

$$\Pr(Y = 2) = \Pr(X = 0, Y = 2) + \Pr(X = 1, Y = 2) + \Pr(X = 2, Y = 2) = 0.00 + 0.05 + 0.05 = 0.10$$

2.2.4. Conditional Probability (X when given Y)

The probability that something will happen in relation to knowledge we already have about another correlating event. Joint probability $\Pr(X, Y)$ and conditional probability $\Pr(X|Y)$ are related in the following way:

$$\Pr(\text{what we want to know} | \text{what we know}) = \frac{\Pr(\text{what we want to know})}{\Pr(\text{what we know})}$$

$$\Leftrightarrow \Pr(X, Y) = \Pr(X|Y) * P(Y) \Leftrightarrow \Pr(Y|X) = \frac{P(X,Y)}{P(X)}$$

Example: Probability of moderate rain (what we want to know) given small clouds (what we know)

$$\Pr(Y = 2 | X = 1) = \frac{\Pr(X=1,Y=2)}{P(X=1)} = \frac{0.05}{0.21+0.07+0.05+0.02} = \frac{0.05}{0.35} \approx 0.14$$

2.3. TWO-STEP EXPERIMENTS

Example: Consider a box with three different coins, a red, a blue and a green one. The red coin is fair. The others have different probabilities for head/tail.

- Red coin R: $P(\text{head}) = 0.5, P(\text{tail}) = 0.5$
- Blue coin B: $P(\text{head}) = 0.7, P(\text{tail}) = 0.3$
- Green coin G: $P(\text{head}) = 0.1, P(\text{tail}) = 0.9$

We now do a **two-step experiment**:

- **Step 1:** Pick a random coin from the box.
- **Step 2:** toss the coin and observe the outcome.

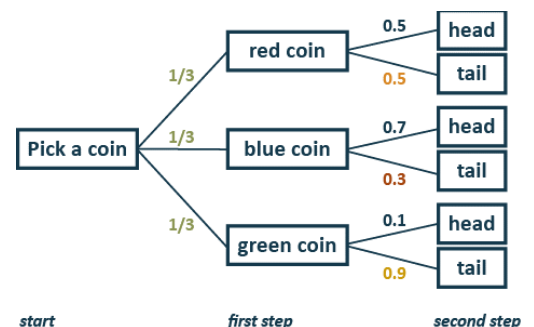
What is the probability to observe “tail”?

2.3.1. Tree Diagram

Tree diagrams are a probabilistic model that explains how data is generated. They are also a structured visualization of the experiment.

To calculate the probabilities, you just need to multiply along the path.

- $P(\text{red, tail}) = \frac{1}{3} * 0.5 = \frac{1}{6} = 0.1\bar{6}$
- $P(\text{blue, tail}) = \frac{1}{3} * 0.3 = \frac{3}{30} = 0.1$
- $P(\text{green, tail}) = \frac{1}{3} * 0.9 = \frac{9}{30} = 0.3$



The probability of observing tail is therefore: $\frac{1}{6} + \frac{3}{30} + \frac{9}{30} = \frac{17}{30} = 0.5\bar{6}$

To generalize: The first step can be written as $P(X)$ and the second as $P(Y|X)$. So, the full calculation can be written as $P(X, Y) = P(Y|X) * P(X)$

The same can be shown in a table of joint probabilities.

	red coin	blue coin	green coin	marginal $P(\text{side})$
head	$\Pr(\text{red, head}) = 0.1\bar{6}$	0.2 $\bar{3}$	0.0 $\bar{3}$	0.4 $\bar{3}$
tail	$\Pr(\text{red, tail}) = 0.1\bar{6}$	0.1	0.3	0.5 $\bar{6}$
marginal $P(\text{coin})$	0.3	0.3	0.3	1

2.4. BAYES RULE

Bayes theorem lets us **“invert” the direction of the tree model**. From observed outcomes, we can make probabilistic statements about the **“hidden causes”** of the observation. The main goal is to **update the probability of an event based on prior knowledge**. $H = \text{Hypothesis}, E = \text{Evidence}$

$$P(X, Y) = P(Y, X) = P(Y|X)P(X)$$

$$\Pr(H|E) = \frac{\Pr(E|H) * \Pr(H)}{\Pr(E)} \Leftrightarrow \text{Posterior} = \frac{\text{Likelihood} * \text{Prior}}{\text{Normalizer (all of the likelihoods = marginal probability)}}$$

Example 1: If we observe tail, which coin was drawn in step 1?

We can formulate this question in terms of probabilities.

Prior: 1/3 , **Likelihood:** 0.5 for red, 0.3 for blue, 0.9 for green

$$\begin{aligned}
 - P(H = \text{red} \mid E = \text{tail}) &= \frac{P(E=\text{tail} \mid H=\text{red}) * P(H=\text{red})}{P(E=\text{tail})} = \frac{0.5 * 1/3}{1/6 + 3/30 + 9/30} = \frac{1/6}{17/30} = \frac{30}{17*6} \approx 0.294 \\
 - P(H = \text{blue} \mid E = \text{tail}) &= \frac{P(E=\text{tail} \mid H=\text{blue}) * P(H=\text{blue})}{P(E=\text{tail})} = \frac{0.3 * 1/3}{1/6 + 3/30 + 9/30} = \frac{1/10}{17/30} = \frac{30}{17*10} \approx 0.176 \\
 - P(H = \text{green} \mid E = \text{tail}) &= \frac{P(E=\text{tail} \mid H=\text{green}) * P(H=\text{green})}{P(E=\text{tail})} = \frac{0.9 * 1/3}{1/6 + 3/30 + 9/30} = \frac{3/10}{17/30} = \frac{90}{17*10} \approx 0.529
 \end{aligned}$$

The result is a **posterior distribution**. It is the result of updating the prior distribution with the evidence / data / observation.

Example 2: If we flip the same coin 3 times and observe **tail, head, head**. Which coin was drawn?

For this example, we need to **repeatedly apply Bayes rule**. We use the first observation to calculate the first Posterior Distribution. The Posterior then becomes the new Prior in the second application. To calculate the second posterior Distribution (getting tail & head), we also need to adjust the normalizer with the first posterior.

The **first application** is the same as in Example 1.

Second application

Prior: 0.294 for red, 0.176 for blue, 0.529 for green, **Likelihood:** 0.5 for red, 0.7 for blue, 0.1 for green

$$\begin{aligned}
 - P(H = \text{red} \mid E = \text{head}) &= \frac{P(E=\text{head} \mid H=\text{red}) * 0.294}{P(E=\text{head})} = \frac{0.5 * 0.294}{0.294*0.5 + 0.176*0.7 + 0.529*0.1} = \frac{0.147}{0.323} \approx 0.455 \\
 - P(H = \text{blue} \mid E = \text{head}) &= \frac{P(E=\text{head} \mid H=\text{blue}) * 0.176}{P(E=\text{head})} = \frac{0.7 * 0.176}{0.294*0.5 + 0.176*0.7 + 0.529*0.1} = \frac{0.123}{0.323} \approx 0.381 \\
 - P(H = \text{green} \mid E = \text{head}) &= \frac{P(E=\text{head} \mid H=\text{green}) * 0.529}{P(E=\text{head})} = \frac{0.1 * 0.529}{0.294*0.5 + 0.176*0.7 + 0.529*0.1} = \frac{0.0529}{0.323} \approx 0.163
 \end{aligned}$$

Third application

Prior: 0.455 for red, 0.381 for blue, 0.163 for green, **Likelihood:** 0.5 for red, 0.7 for blue, 0.1 for green

$$\begin{aligned}
 - P(H = \text{red} \mid E = \text{head}) &= \frac{P(E=\text{head} \mid H=\text{red}) * 0.455}{P(E=\text{head})} = \frac{0.5 * 0.455}{0.455*0.5 + 0.381*0.7 + 0.163*0.1} = \frac{0.228}{0.510} \approx 0.444 \\
 - P(H = \text{blue} \mid E = \text{head}) &= \frac{P(E=\text{head} \mid H=\text{blue}) * 0.381}{P(E=\text{head})} = \frac{0.7 * 0.381}{0.455*0.5 + 0.381*0.7 + 0.163*0.1} = \frac{0.268}{0.510} \approx 0.523 \\
 - P(H = \text{green} \mid E = \text{head}) &= \frac{P(E=\text{head} \mid H=\text{green}) * 0.163}{P(E=\text{head})} = \frac{0.1 * 0.163}{0.455*0.5 + 0.381*0.7 + 0.163*0.1} = \frac{0.016}{0.510} \approx 0.032
 \end{aligned}$$

This means, it is most likely that the blue coin was drawn (52%)

3. LINEAR REGRESSION

Linear Models are the **simplest model** to explain a **relationship** between “Input” and “Output”. Standard method to find an optimal linear model.

Interpretation: Understand if some input has an effect on the output.

Example: Is there a relationship between smoking cigarettes and the risk of lung cancer?

Prediction: Given a new x , use the model to predict / estimate the y .

Example: x is smoking rate, y is death rate

Linear regression belongs to **supervised learning**: The algorithm learns a linear relationship between x and y , both are given.

3.1. MODEL

A model is a **mathematical function that “explains the data”**.

$$y_i \approx f(x_i), \quad y_i = f(x_i) + \varepsilon_i$$

ε_i is **“unexplained noise”**. It is assumed that ε_i follows a normal distribution (Bell Curve/Glockenkurve). The function f can be simple or very complicated. The goal of ML is to **find the model which explains the data** (as good as possible). Also, instead of approximating y_i , we calculate an estimate $\hat{y}_i$ of the usually unknown y_i .

In linear regression, we **only consider a linear relationship** between the input and output. There are therefore only **two free parameters, a and b** . The goal is to identify a and b for which the linear model **“best explains the data”**. a is usually called **slope**, b the **intercept**.

$$\hat{y}_i = ax_i + b$$

How to find out if a model is good or bad?

3.2. MEAN SQUARED ERROR (MSE)

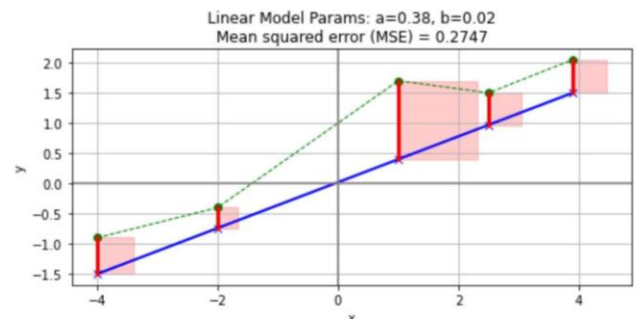
This is the loss we want to minimize.

$$\hat{y}_i = ax_i + b$$

$e_i = y_i - \hat{y}_i$ (e_i = the residual Difference between our estimate and the actual value)

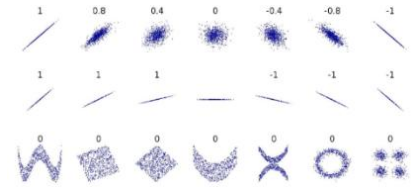
$$E = \frac{1}{2N} \sum_{i=1}^N e_i^2 = \frac{1}{2N} \sum_{i=1}^N (y_i - (ax_i + b))^2$$

E (Error) is the Sum of the areas of the residual Squares (red) divided by two times the number of squares.



3.2.1. Pearson Correlation Coefficient (r)

Most common way of measuring a linear correlation. It is a number between -1 and 1 that measures the strength and direction of the relationship between two variables.



r	Correlation type	Interpretation
$0 < r < 1$	Positive Correlation	Both variables change in the same direction . <i>Positive Steigung</i>
0	No correlation	There is no linear relationship between the variables.
$-1 < r < 0$	Negative Correlation	The 2. variable changes in the opposite direction . <i>Negative Steigung</i>

3.3. MULTIPLE LINEAR REGRESSION

Add **more “explaining factors”** to the model, since most of the time, multiple factors contribute to the result to different degrees: $y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$

Same concept, but different notation / indexing: $Y_i = \alpha + \beta_1x_i^{(1)} + \beta_2x_i^{(2)} + \dots + \beta_nx_i^{(n)}$

Idea: a single “dependent” variable y is **explained by multiple independent variables** x . To be able to change the importance of each variable, we also add a **weight** w_n/β_n . Example: y_i is an observed/measured quantity. Example: blood pressure. $x_1 \dots x_n$ are “factors” like age, weight, sex, ... $w_1 \dots w_n$ are weights. How much does each factor x explain the outcome y ?

A variant of multiple linear regression is polynomial linear regression, where each variable is an exponent $y = w_1x_1 + w_2x_2^2 + w_3x_3^3 + \dots + w_px_p^d + w_0$

3.3.1. Matrix Notation

Dataset: n points (x, y) where x is a vector with p features (=dimensions).

The model: $\hat{y}_i = \beta_0 + \beta_1x_{i1} + \beta_2x_{i2} + \dots + \beta_px_{ip}$ can be written much more compactly if we use matrix notation.

$y = X\beta + \beta_0$, where X = Datapoints, β = Weights, y = Estimates:

$$X = \begin{bmatrix} X_{11} & X_{12} & \dots & X_{1p} \\ X_{21} & X_{22} & \dots & X_{2p} \\ \dots & \dots & \ddots & \vdots \\ X_{n1} & X_{n2} & \dots & X_{np} \end{bmatrix}, \beta = \begin{bmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{bmatrix}, y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

4. GRADIENT DESCENT

Gradient Descent is a fundamental **optimization algorithm**. When an AI is “training” or “learning”, this means that an algorithm is performing some sort of optimization, like **minimizing the loss function**. It only works if we can express the loss function as a differentiable function, this is not always the case.

The **gradient of a function** is the **collection of all its partial derivatives** organized in a vector.

$$\text{Gradient of } E = \begin{bmatrix} \frac{\partial E}{\partial a} \\ \frac{\partial E}{\partial b} \end{bmatrix} = \begin{bmatrix} \frac{1}{N} \sum_{i=1}^N (y_i - (a * x_i + b))(-x_i) \\ \frac{1}{N} \sum_{i=1}^N (y_i - (a * x_i + b))(-1) \end{bmatrix}, \begin{bmatrix} a \\ b \end{bmatrix}_{t+1} = \begin{bmatrix} a \\ b \end{bmatrix}_t - \alpha \begin{bmatrix} \frac{\partial E}{\partial a} \\ \frac{\partial E}{\partial b} \end{bmatrix} \Big|_{\begin{bmatrix} a \\ b \end{bmatrix}_t}$$

Example: $a = 0.6, b = 0.4, \frac{\partial E}{\partial a} = +0.2, \frac{\partial E}{\partial b} = -0.3, \alpha = 0.1 \rightarrow \begin{pmatrix} 0.6 \\ 0.4 \end{pmatrix} - 0.1 * \begin{pmatrix} 0.2 \\ -0.3 \end{pmatrix} = \begin{pmatrix} 0.58 \\ 0.43 \end{pmatrix}$

It will **always point in the direction where there is the greatest increase** in the function, in our case the loss. Since we want to minimize our loss, we need to invert our gradient (descending the gradient). Gradient Descent is an iterative operation, so we

run GD again until the result converges (no/very small change). We can also set a convergence threshold, if the last move is smaller than this, we also end the GD.

The gradient descent follows these steps:

1. Pick **a random point** w in the function, this is the **starting point**. This point is represented by the row vector $w = [x_0 \ x_1 \ x_2 \ \dots \ x_n]$
2. While the gradient hasn't converged (iterative part of the algorithm)
 - a. **Compute the negative gradient at w to all other data points and pick the one with the greatest descent.**
 - b. **Move the location by the result of 2a.**
3. **Repeat** until you have found the minimum or reached the **convergence threshold**. If, compared to the previous iteration, the new gradient of point w has not changed more than the convergence threshold, the algorithm has converged.

$$\underbrace{w^{(t+1)}}_{\text{position of next iteration}} = \underbrace{w^{(t)}}_{\text{position of previous step}} - \underbrace{\alpha \nabla f(w^{(t)})}_{\text{learning rate} \times \text{step}}$$

Learning rate α

The learning rate alpha is the **size of the step** Gradient Descent takes all the way until it reaches a minimum, and it directly impacts the performance of the algorithm. When it's **too big**, you're taking big steps, so you may step over the minimum and **never reach it**. When the learning rate is **too small**, the algorithm might **take a long time** to find the minimum.

Limitations of Gradient Descent

- Calculating derivatives for the entire dataset is **time consuming**.
- **Memory** required is proportional to the size of the dataset.

4.1. STOCHASTIC GRADIENT DESCENT (SGD)

It is a probabilistic approximation of Gradient descent. It is an approximation because, at **each step**, the algorithm **calculates the gradient for one data point picked at random**, instead of calculating the gradient for the entire dataset. This represents a significant performance improvement. But because the gradient is not computed for the entire dataset, and only for one random point on each iteration, the updates have a **higher variance**. This makes the **cost function fluctuate more** on each iteration, making it harder for the algorithm to converge.

$$\underbrace{w^{(t+1)}}_{\text{position of next iteration}} = \underbrace{w}_{\text{position of previous step}} - \underbrace{\alpha \nabla f_i(w^{(t)})}_{\text{learning rate} \times \text{observation } i \times \text{step}}$$

4.1.1. Batch-Gradient-Descent

Often, **batch-gradient-descent** is used which **uses random subsets** (or batches) **instead of one random point**. This is more efficient. Typical batch sizes: 32, 64, ..., 1024 samples.

4.1.2. Annealed Stochastic Gradient Descent

The **learning rate α gets adapted**. Starts the algorithm with a **large** learning rate and then **reduces** it over time for example by multiplying it at each iteration by a **decay_factor** like 0.99. Typically, there's a lower bound where the learning rate doesn't decrease any further.

5. REGULARIZATION

Too complex models generalize badly. Too simple models may miss information and perform sub-optimally. Those two observations are related by the bias-variance trade-off (aka bias-variance dilemma). With regularization, we can **constrain the learning process**.

5.1. MODEL TESTING

The model must perform well on new, unseen inputs which means it must generalize well to new data.

In-sample Error (aka Training Error): Difference between the predictions and the actual results on the same data the model was trained on. It is possible to find a model which perfectly fits the data. When a model has an MSE of 0, the data was probably overfitted. Overfitted models perform great on the training data, but badly on new data. So ideally, the training error should be minimal, but not zero.

Out-of-Sample Error (aka Test Error): Difference between the predictions and the actual results on different data the model wasn't trained on. Difference between the predictions and the actual results on different data the model wasn't trained on. If we use our model on new data sample $(x_{\text{unseen}}, y_{\text{unseen}})$ to test how well it predicts for new data, we can calculate the error of the prediction using $\hat{y}_{\text{unseen}} = h(w, x_{\text{unseen}})$ and y_{unseen} . This is the out-of-sample error.

The goal is to **learn a model** from data that **generalizes well** to new data. A "good" model has a low generalization error. So **both errors** should be **as low as possible**.

5.1.1. Splitting Technique

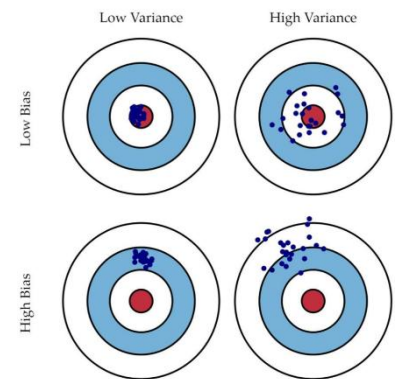
We can't calculate the generalization error, because we do not have "new data" to test our model. We can only estimate it by **splitting the data** we are given into two sets: The training and test set. A common split ratio is 80% / 20%, so we have most of the data in the training set, while still having enough data to test with. The data in the test-set does not get used during fitting.

- **Training Phase:** Fit the model to the training set. This minimizes the in-sample error.
- **Evaluation:** Evaluate the model using the test-set. This gives us an estimate of the generalization error.

5.1.2. Bias-Variance Trade-Off

By analyzing the prediction error mathematically, one can decompose it into two terms: bias (average difference between predicted and actual values) and variance (difference between different runs of a model). The expression "high bias" is used in the sense of "a too simple model for the given data"

- **High Variance:** Not Precise Estimates are spread out
- **Low Variance:** Precise Estimates are clustered together
- **High Bias:** Not Accurate, high training error Model missed relevant relations between features and outputs
- **Low Bias:** Accurate Estimates are close to the correct result



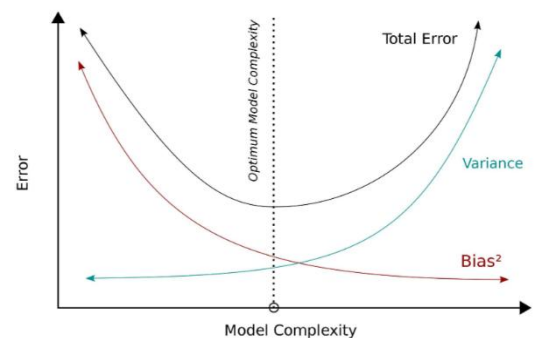
Too Simple Model: High Bias, Low Variance (underfitting)

A very simple model (i.e. simple linear regression) cannot do better than finding a line, no matter how much data we give it to learn. Such a model imposes a **high bias**. It **fails to learn the underlying structure** of the data. By imposing a high bias, we assume a "simple world" and reduce the space of what can be learned.

The flip-side of a too simple model is that it is relatively stable. With another test-sample, the model would find a very similar line. So for a **change in data**, we would fit **almost the same model**. This is the meaning of **low variance**.

Too Complex Model: Low Bias, High Variance (overfitting)

Low bias: A more complex model is **less restrictive**. It can better "explain" the data. A **high variance** means that for a **different set** of data points, the **model** could be **very different**.



The trade-off

Higher bias implies lower variance, lower bias implies higher variance. In practice, we do not directly care about the bias, we just want a low variance (reliable predictions). But we can only build a model as complex as the data permits. We therefore have to find an optimal balance between bias and variance.

5.2. REGULARIZATION

With regularization, we reduce the number of polynomial degree to **avoid overfitting** or increase the degree to **avoid underfitting**. It does this by decreasing variance at the cost of increasing bias. This in turn decreases the training accuracy, but increases generalizability. Regularization **adds a Constraint** to the model, rather, its Optimizer, to achieve this.

- **Measure of performance:** regression error (MSE) how well the model predicts data
- **Measure of complexity:** regularization term control the complexity of the model

We want to Minimize the regression error + regularization term. It is common to have two separate functions: An optimization function for the optimizer (Gradient Descent) and a performance evaluation function to evaluate the error.

5.2.1. How to express Model Complexity

The complexity of a model can be expressed by multiple parameters: degree of polynomial, number of features and size of coefficients. There are two different ways to express the complexity:

- L2-Norm (Euclidean Norm, Sum of weights): $\sum_{j=1}^p w_j^2$
- L1-Norm (Manhattan distance / Taxicab norm, Sum of absolute weights, «Häuschen zählen»): $\sum_{j=1}^p |w_j|$

We add one of these constraints to the optimizer to find the best weights for our model.

5.2.2. Ridge

Ridge uses the L2-Norm (Euclidean Norm). Minimize:

$$MSE_{ridge}(X, h(w, x)) = \frac{1}{2N} \sum_{j=1}^N (y_i - h(w, x_j))^2 + \lambda * \sum_{j=1}^p w_j^2 = \text{MSE} + \text{Hyperparameter} * \text{L2-Norm}$$

Example Calculation of L2-Norm: Point 1 is at $x = 3, y = 3$ and Point 2 is at $x = 2, y = 2$. So, Lambda gets multiplied by $\sqrt{2}$.

$$d_E(x_1, x_2) = \sqrt{\sum_{i=1,p} (x_{1,i} - x_{2,i})^2} \Rightarrow d_E(1,2) = \sqrt{(3-2)^2 + (3-2)^2} = \sqrt{2}$$

L2 regularization is not robust to outliers. The squared terms will blow up the differences in the error of the outliers. The regularization would then attempt to fix this by penalizing the weights.

5.2.3. Lasso (least absolute shrinkage and selection operator)

Lasso uses the L1-Norm (Manhattan distance). Minimize:

$$MSE_{lasso}(X, h(w, x)) = \frac{1}{2N} \sum_{j=1}^N (y_j - h(w, x_j))^2 + \lambda * \sum_{j=1}^p |w_j| = \text{MSE} + \text{Hyperparameter} * \text{L1-Norm}$$

Lasso can force the weights to 0, unlike Ridge. It enables us to perform feature selection, making certain weights 0. $w_i = 0$ means that x_i is not relevant.

When we have highly correlated features *i.e. number of rooms and house area size*, the L1 norm would select only 1 of the features from the group of correlated features in an arbitrary nature, which is something that we might not want.

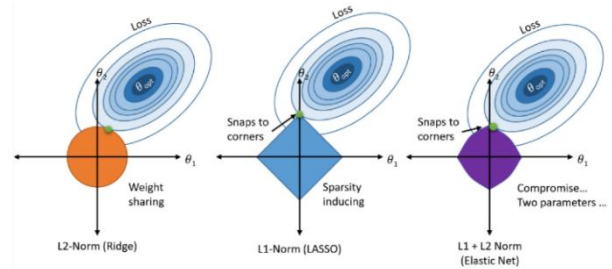
Example Calculation of L1-Norm: Point 1 is at $x = 3, y = 3$ and Point 2 is at $x = 2, y = 2$.

$$d_M(x_1, x_2) = \sum_{i=1}^p |x_{1,i} - x_{2,i}| \Rightarrow d_M(1,2) = (3-2) + (3-2) = 2$$

To test if you have too many features, you can use lasso regression to see if it eliminates any features.

Lambda (λ)

λ is a hyperparameter, it does not belong to the optimization process as such. It is varied to find the best fit. **When it is zero, the $MSE_{ridge/lasso}$ is just the normal MSE.** As λ gets larger, we are enforcing the weights to be smaller by constraining the squared sum of weights more and more. **Increasing λ makes the model simpler, increases bias and reduces variance.**



6. CROSS VALIDATION

Hyperparameter: Specifies details of the learning process such as parameters of the optimizer (*learning rate, type of gradient descent, regularization parameters (L1, L2, Lambda) etc.*)

3-way holdout: Split data in **training-data**, **validation-data** and **test-data**.



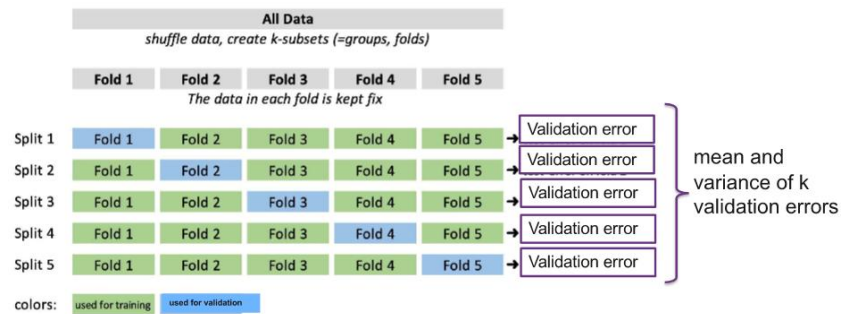
- Train the model with the **training-data**, multiple models with different hyperparameters
- Validate the trained models with the **validation-data**. Take the hyperparameters with the best score.
- Train another model with the best hyperparameters and the **training + validation data**. Test this model with the **test-data**.
- Optional: Train the best model again with all the data.

Problems

- Training error is **too optimistic** about generalization (overfitting).
- Test error is **unbiased** but can be **too pessimistic**. The generalization error is calculated only on certain 20% set, which could lead to the model only being optimized for those 20% of the data.
- Test and training data may **not be representative** of the general/overall dataset.

6.1. K-FOLD CROSS VALIDATION

Cross validation is a technique to address these problems. It is an extension of the holdout method. With the k-fold cross-validation, **the data is split into k folds**. Then the **train/validation process is repeated k-times**. Each fold participates in k-1 training phases and is used once for validation. The folds can be overlapping.



We can use cross-validation to **obtain a better estimate of the generalization error**. This is also known as model evaluation. **After** k-fold cv, we can **train the model on the complete data** using the **fixed hyperparameters** and deploy that model. If $k = 2$, the Model is split into 50% training data and 50% test data. If $k = N$ # of data in the dataset, only one value is used for testing on every split (**LOOCV** – Leave one out cross validation). Typical values for k are 5,10 or N . It is better to **apply the preprocessing pipe-line** (e.g. standardization) to **each split, not only once in the beginning** for the whole dataset. Otherwise, the results may be distorted. We can also find the best hyperparameter by running KFCV for each set of hyperparameters and pick the set with the smallest mean validation error.

7. FEATURE SCALING

Feature scaling is a method to **normalize the range of independent variables** of data. If for example, you have multiple independent variables like age, salary and height with ranges 18-100 years, 25'000 – 75'000 and 1-2 meters, feature scaling transforms them all to be in the **same range**. If the range differences are too big, small changes in the weights of large features have a huge impact on the MSE, while weights of small features need huge changes to affect the MSE.

Regularization penalizes larger coefficients more than the smaller ones. **Standardization puts all the features on equal footing**.

7.1. SKLEARN STANDARDSCALER

Rescales a dataset to have a mean of 0 and a standard deviation of 1.

$$x_{std} = \frac{x - X_{mean}}{s}, s = \sqrt{\frac{1}{N+1} \sum_{i=1}^N (x_i - X_{mean})^2}$$

Example: Data Points x_i : 2,4,4,4,5,5,7,9. $X_{mean} = 40/8 = 5$. Sample variance $s^2 = 4.57$, Sample Std. Deviations $s = 2.138$. Standardization of 2 = $\frac{2-5}{2.138} = -1.4$

Raw Data	Normalized Data
2	-1.403
4	-0.468
4	-0.468
4	-0.468
5	0.000
5	0.000
7	0.935
9	1.871

8. CLASSIFICATION AND LOGISTIC REGRESSION

- **Binary Classification:** Only two classes *true/false*. Example: Epileptic seizure or healthy state?
- **Multi-Class Classification:** More than two classes. Example: Match is won, it's a tie, Match is lost

8.1. LOGISTIC REGRESSION

Used for **binary classification** (Yes/No, Spam/no spam). Linear regression is not usable for binary classification, because it is linear. Even with a threshold, the function does not work well with only two outputs, it outputs continuous values. If we input outlier data, the whole model gets distorted. The MSE does not work. So we need a **probabilistic function** like the sigmoid. We want a cost function over the probability that the data point belongs to a certain class.

8.1.1. Multi class logistic regression

One vs rest

Train a single classifier for each class with the samples of it as positive and all other as negative (*Sunny vs not sunny, cloudy vs not cloudy*). Applied to an unseen sample, the classifier with the highest p is the class.

One vs one

Train classifiers to distinguish between each pair of classes (*sunny, cloudy*), (*sunny, rainy*), (*sunny, snowy*)... Applied to an unseen sample, combine all results to produce the final classification.

8.1.2. Sigmoid function

$$\text{sigmoid}(z) = \frac{1}{1+e^{-z}}$$

$$z = h(w, x) = w_1x_1 + w_2x_2 + w_3x_3 + \dots$$

This is where our features (data x) enters the calculation. The w 's are unknown. That's what needs to be **learned**.

Why sigmoid? Because of the odds ratio *how many times more likely x will be accepted vs. against x*

$$\text{will be rejected: } \text{odds}(p) = \frac{p}{1-p} = \frac{\Pr(y=1)}{\Pr(y=0)}$$

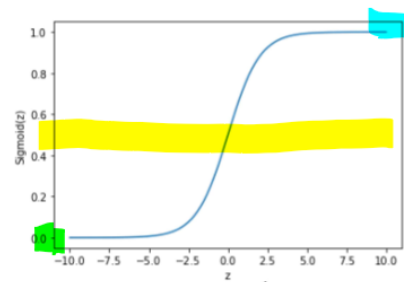
We can write the estimated probability as (formula for exercises)

$$\Pr(y = 1|x; W) = \Pr(y = 1|x) = p(x) = \frac{1}{1+e^{-(W^T x)}} = \frac{1}{1+e^{-(w_1x_1+w_2x_2+w_3x_3+w_4)}}$$

8.1.3. Maximum Likelihood (Cost function of logistic regression)

Given all the data points (X, Y) , we want to maximize the probability that all the predictions are correct. The objective of training is to set the coefficients W so that p (the prediction) is close to 1 when the actual data $y = 1$, (p_i) and close to 0 when $y = 0$, ($1 - p_i$). This can be calculated using gradient descent. The cost of $\log(p)$ & $\log(1 - p)$ is small on wrong and large on correct predictions.

$$\text{Minimize cost}(W) = -\frac{1}{N} \sum_{i=1}^N (y_i * \log(p_i) + (1 - y_i) * \log(1 - p_i))$$



$$\text{sigmoid}(0) = \frac{1}{1+e^{-0}} = 0.5$$

$$\text{sigmoid}(\infty) = \frac{1}{1+e^{-\infty}} = 1$$

$$\text{sigmoid}(-\infty) = \frac{1}{1+e^{\infty}} = 0$$

9. CLASSIFIER EVALUATION

How to calculate accuracy and error from the confusion matrix?

- **Accuracy:** How often is the classifier correct: $\frac{TP+TN}{\text{number of all inputs}}$
- **Error:** How often is the classifier wrong: $\frac{FP+FN}{n}$

These metrics can be misleading, as a model that always predicts that a patient is healthy can still have a very high accuracy, even if all sick patients have been misclassified. Example: 10/1000 patients are sick, everyone is classified as healthy, accuracy is still 99%

It **depends on the objective** if a False Positive or a False Negative is worse. (sickness: false negative, Spam: false positive)

		Predicted condition	
		Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP),	False negative (FN),
	Negative (N)	False positive (FP),	True negative (TN),

9.1. RECALL/SENSITIVITY (TRUE POSITIVE RATE)

Useful when false negatives are worse. Among the positive ground truth samples (P), how many did we correctly classify? If you have no false negatives because you have no negatives, you can fool recall.

$$\text{Recall} = \frac{TP}{TP+FN} \text{ (all with label positive)}$$

9.2. PRECISION

Useful when false positives are worse. Among the predicted positives (PP), how many were correctly classified?

$$\text{Precision} = \frac{TP}{TP+FP} \text{ (predicted positives)}$$

9.3. F-SCORE

Combining precision and recall. In real life, false negatives and false positives are bad. So we need the harmonic mean of Precision P and Recall R . If **both** P and R are **high**, the F_1 score is **high**. If **one of them** is **low**, the F_1 score is **also low**.

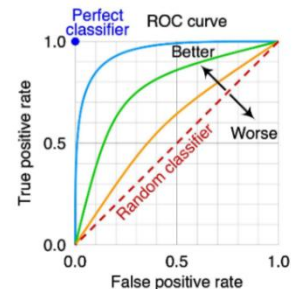
$$F_1 = \frac{2PR}{P+R} = \frac{2}{\frac{1}{P} + \frac{1}{R}}$$

There is also a combined metric $F_\beta = \frac{(1+\beta^2)*P*R}{\beta^2*(P+R)}$. β acts as a dial to decide the emphasis between precision and recall. $\beta = 1 \rightarrow F_\beta = F_1$ (Both equally important), $\beta = 0 \rightarrow F_\beta = P$ (Recall not important), $\beta = \infty \rightarrow F_\beta = R$ (Precision not important).

9.4. THRESHOLD

There is *no universal solution*, different goals require different thresholds.

1. Train your machine learning model
2. Use the trained model to make predictions on your test set, so that each example has a classification probability between 0 and 1.
3. Using a variety of threshold values, *convert the predicted probabilities to predicted classes*. Calculate True positive rate ($TPR = P$) and False positive rate ($FPR = \frac{FP}{FP+TP}$). *Different thresholds result in different TPR and FPR.*
4. *Plot a curve* of TPR vs FPR for the different thresholds.



9.4.1. Receiver Operating Characteristics (ROC)

A ROC space is defined by FPR and TPR as x and y axes, respectively, which depicts *relative trade-offs between true positive and false positive*.

Area under the curve (AUC) shows *how well* the TPR and FPR is looking in the aggregate. The *greater* the *area* under the curve, the *greater* the *quality* of the model. If the AUC is < 0.5 , the model is useless, because it is worse than just randomly assigning classes.

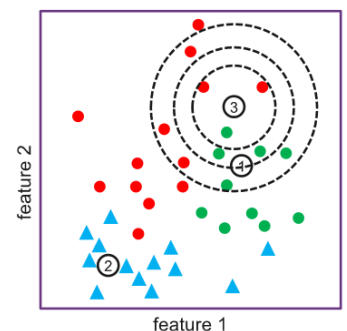
10. K-NEAREST-NEIGHBOURS (KNN)

If a simple line perfectly separates the classes, then the classes are said to be *linearly separable*. However, what to do when the classes are not linearly separable? Logistic regression is possible, but very inconvenient. That's where KNN comes into play. Basic idea: *"A datapoint is known by the company it keeps"*.

Given a test data point, KNN computes k nearest neighbours of it and returns the most frequent class of the k neighbours.

Example in image:

	$k = 3$	$k = 5$	$k = 10$
Sample 1	green	green	green
Sample 2	blue	blue	blue
Sample 3	red	green	red or green



10.1. KNN DETAILS

- Load the training and test data
- *Chose value of k* (the number of nearest neighbours to consider for classification) Should be tuned based on validation error. $k = 1$ results in overfitting, a increase in k leads to smoother boundaries, $k = n$ results in underfitting
- *Chose a distance metric* (Euclidean, Manhattan, cosine, Minkowski, ...)
- For each test data points x_{test} :
 - For all training data x_{train} , *calculate the distance* $d(x_{test}, x_{train})$ with your distance metric
 - *Sort* training data in the ascending order of the distance
 - *Choose the first k* data points from the sorted training data
 - *Choose the most frequently occurring class* from the k data points as the classification result.

Advantages of KNN: Easy and simple machine learning model. Few hyperparameters to tune k & distance metric.

Disadvantages: k should be wisely selected, Large computation cost during runtime if dataset is large. Not efficient for high dimensional datasets, proper scaling should be provided.

10.2. DISTANCE METRICS

Given $x_1 = (x_{1,1}, x_{2,1}, \dots, x_{p,1})$ and $x_2 = (x_{1,2}, x_{2,2}, \dots, x_{p,2})$. *Example:* $x_1 = (1,1)$, $x_2 = (2,2)$, $p = 2$ (hyperparameter used for minkowski distance, normally 1 or 2)

Cosine distance

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\text{dot product of the vectors}}{\text{Length of the vectors multiplied}} = \frac{1 \cdot 2 + 1 \cdot 2}{\sqrt{1^2 + 1^2} \cdot \sqrt{2^2 + 2^2}} = \frac{4}{\sqrt{2} \cdot \sqrt{8}} = \frac{4}{\sqrt{16}} = 1$$

Manhattan distance

$$d_{MH}(x_1, x_2) = \sum_{j=1}^p |x_{i,1} - x_{i,2}| = (|1 - 2|) + (|1 - 2|) = 1 + 1 = 2$$

Euclidean distance

$$d_E(x_1, x_2) = \sqrt{\sum_{j=1}^{i=p} (x_{i,1} - x_{i,2})^2}, \sqrt{(1-2)^2 + (1-2)^2} = \sqrt{2}$$

Minkowski distance:

$$d_{MK}(x_1, x_2) = (\sum_{i=1}^p (|x_{i,1} - x_{i,2}|^p))^{\frac{1}{p}}, (|1-2|^2 + |1-2|^2)^{\frac{1}{2}} = 2^{\frac{1}{2}} = \sqrt{2}$$

11. NAÏVE BAYES CLASSIFIER

Naïve Bayes is a generative method for classification (*it generates something*) based on Bayes' Theorem (See page 4). It assumes that **all the features that predict the target value are independent**. It describes the **probability of an event based on a prior knowledge** of conditions.

Naïve Bayes is good when the dataset is small and there is no training phase. It is used extensively when data contains categorical features, but it's not used much in numerical features. However, we can use binning to create categories.

$$\Pr(y|X) = \frac{\Pr(X|y) * \Pr(y)}{\Pr(X)} = \frac{P(x_1|y) * P(x_2|y) * \dots * P(x_3|y) * P(y)}{P(x_1) * P(x_2) * \dots * P(x_n)}$$

Assume we have a bunch of emails we want to classify as spam or not spam. How to calculate $\Pr(\text{spam} | \text{"Hurry"})$?

We can simply take each word as a separate feature.

All the words: **Hurry**, **Sale**, **Tomorrow**, **Rain**, **Price**, **Workshop**

So "Hurry Sale Tomorrow" can be encoded as $1 = \text{spam}$, $0 = \text{not spam}$:

$$x_{\text{hurry}} = 1, x_{\text{sale}} = 1, x_{\text{tomorrow}} = 1, x_{\text{rain}} = 0, x_{\text{price}} = 0, x_{\text{workshop}} = 0$$

Now when an email contains "hurry", would it be classified as spam or ham? We can find out by calculating the probability that the email is spam given that it contains "hurry".

$$\Pr(\text{spam} | x_{\text{hurry}} = 1) = \frac{\Pr(x_{\text{hurry}}=1 | y=1) * \Pr(y=1)}{\Pr(x_{\text{hurry}}=1)} \quad (y = 1 \text{ means spam})$$

$$- \Pr(y = 1) = \Pr(\text{spam}) = \frac{\text{\#entries in the data set that are spam}}{\text{\#size of data set}} = \frac{2}{4} = \frac{1}{2}$$

$$- \Pr(x_{\text{hurry}} = 1) = \Pr(\text{"hurry"}) = \frac{\text{\#entries that contain "hurry"}}{\text{\#size of data set}} = \frac{1}{4}$$

$$- \Pr(x_{\text{hurry}} = 1 | y = 1) = \Pr(\text{"hurry"} | \text{spam}) = \frac{\text{\#occurences that are spam and contain "hurry"}}{\text{\#occurences that are spam}} = \frac{1}{2}$$

Final calculation: $\Pr(\text{spam} | x_{\text{hurry}} = 1) = \left(\frac{1}{2} * \frac{1}{2}\right) / \frac{1}{4} = \frac{1}{4} / \frac{1}{4} = 1 \Rightarrow$ email no. 5, "hurry sale" will be classified as **spam**.

nr.	email header	spam
1	Hurry Sale Tomorrow	1
2	Rain tomorrow	0
3	Sale price tomorrow	1
4	Tomorrow workshop rain	0
5	Hurry sale	?

12. UNSUPERVISED LEARNING - CLUSTERING

When we are given **data without labels** classifier of the data, can we still learn something from the data? **Yes**. Often, the data has some structure. The goal of **unsupervised learning** is to **self-discover patterns** from the data without any training.

A simple example of a **structure in the data** are **clusters**. i.e., the data points which have some shared properties will group together into a cluster.

12.1. CLUSTERING

The goal of clustering is to group n data points into k_c number of clusters. How do we do that? Similar principle as KNN

12.1.1. Naïve K-means

- Let us assume we **know the number of clusters** k_c we want to group the data in.
- Initialize** the value of k **cluster centres** (aka means, centroids) $C_1, C_2, \dots, C_{k_c}$. Usually randomly initialized.
- Find the **squared Euclidean distance** between the **centres** and **all the data points**. **Assign** each data point **to the cluster** of the nearest center.

- Each cluster now potentially has a **new centre** (mean). **Update the centre** for each cluster. The new center is the average of all the data points in the cluster.
- If some **stopping criterion** met, done (like centres do not change anymore, the distance of datapoints to the centre is bigger than a set threshold or a fixed number of iterations has been reached). Else, **go to step 3**.

12.1.2. Evaluate Cluster quality

The number of clusters is a **hyperparameter**. You need at least two clusters, but less than the amount of data points. How can one evaluate the **cluster quality**?

Goal of good clustering: Create clusters so that for each cluster the distance of each cluster member from its center is minimized. There are two approaches to find the optimum.

Inertia or within-cluster sum-of-squares (WCSS)

Sum of squared distances of samples to their closest cluster centre (How far away the points within a cluster are). A **small inertia is desired**.

Example:

	Squared Euclidean distance from red centre	Squared Euclidean distance from green centre
	1.25, 2	2.5, 1
1,3	$(1.25 - 1)^2 + (3 - 2)^2 = 1.0625$	-
1,2	0.0625	-
1,1	1.0625	-
2,1	-	0.25
2,2	0.5625	-
3,1	-	0.25
WCSS	2.75 wide cluster	0.5 tight cluster

Do this **calculation** as you **increase** the amount of clusters. Then draw a **plot** k_c vs inertia. The more clusters we have, the closer the points are to their centers.

The optimal amount of clusters is found at the **"elbow"** of the graph. In this example 3 or 4

Silhouette Score

The silhouette score considers both the **cohesion** a (how far apart the points in a cluster are) and **separation** b (how far apart the clusters are). It provides a value between **-1 and 1** for each data point, with **higher values** indicating **better-defined clusters**. The overall Silhouette Score for a clustering solution is the **average** of these individual scores:

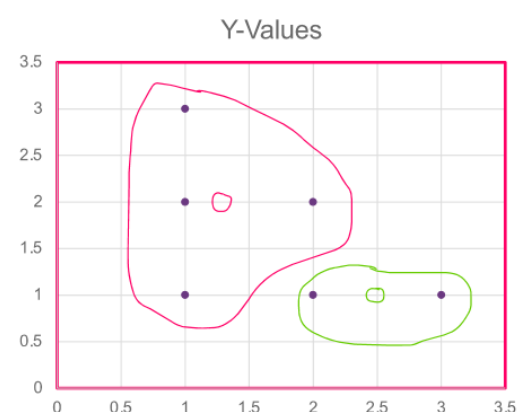
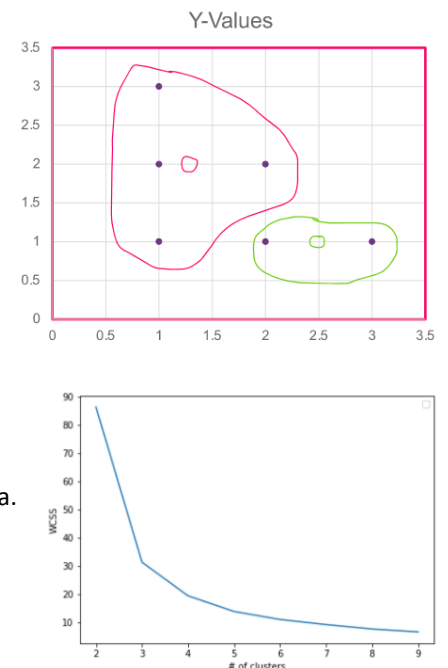
$$(b - a) / \max(a, b)$$

Example: calculation of value a (average of all distances)

Distance from other points in the cluster					
	1,3	1,2	1,1	2,2	a
1,3	—	1	2	$\sqrt{2}$	1.47
1,2	1	—	1	1	1
1,1	2	1	—	$\sqrt{2}$	1.47
2,2	$\sqrt{2}$	1	$\sqrt{2}$	—	1.27
Distance from other points in the cluster					
	2,1		3,1		a
2,1	—		1		1
3,1	1		—		1

Calculation of value b (average of all distances)

Distance from points in the RED cluster					
	1,3	1,2	1,1	2,2	b
2,1	$\sqrt{5}$	$\sqrt{2}$	1	1	1.41
3,1	$\sqrt{8}$	$\sqrt{5}$	2	$\sqrt{2}$	7.42



So the Silhouette Score of (2,1) = $(b - a) / \max(a, b) = (1.41 - 1) / 1.41 = 0.29$. Using WCSS, the elbow suggests $k \approx 3$, while the silhouette score is maximal at $k = 2$. Therefore, a reasonable choice is $k = 2$ or $k = 3$, depending on the desired trade-off.

12.1.3. Performance

The **performance depends** on the random **initialization** of the seeds for the centroids. Some seeds result in a **poor convergence rate**, some can converge to **suboptimal clustering**. If the initial centers are **very close** together, it would take **a lot of iterations** for the algorithm to converge. The best way is to **initialize randomly** and **run multiple times**. If the clusters are **stable**, the clustering is **optimal**.

Features with large values may dominate the distance value. Features with small values will have no impact in the clustering. That's why you should **always employ feature scaling** (normalize values).

12.1.4. Example calculating cluster centre

$$\frac{\text{Sum of } x \text{ coefficients}}{\text{amount of data points in cluster (without centre)}} \quad \frac{\text{sum of } y \text{ coefficients}}{\text{amount of data points in cluster (without centre)}}$$

- **Center of red cluster:** $C_{R_x} = \frac{1+1+0.4}{3} = 0.8$, $C_{R_y} = \frac{3+2+2}{3} = 2.33$
- **Center of green cluster:** $C_{G_x} = \frac{2+2+2.5}{3} = 2.17$, $C_{G_y} = \frac{2+1+1}{3} = 1.33$

Will a new point [0.5, 1] be assigned to the red or the green cluster?

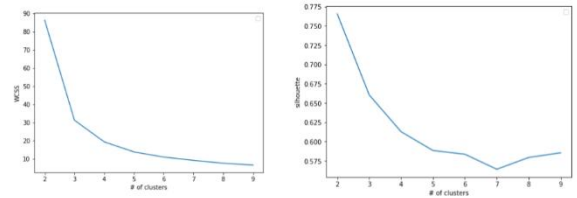
Squared Euclidean distance from red cluster:

$$(0.5 - 0.8)^2 + (1 - 2.33)^2 = 1.85$$

Squared Euclidean distance from green cluster:

$$(0.5 - 2.17)^2 + (1 - 1.33)^2 = 2.99$$

Now that a new point is added to the red cluster, the **center needs to be recalculated**.



WCSS, Elbow Method

Silhouette Score



13. ENSEMBLE

The **combining of multiple weak models** and the aggregation of their results is called ensemble learning. Aggregating results of many weak predictors for a better prediction. Techniques: Voting, Bagging, Boosting.

Ensemble works best, when:

- The weak models are **better than random**.
- The models are **independent from one another** and make uncorrelated errors.
- There is a **sufficient number** of weak learners.
- The models are **not trained on the same data** otherwise likely to make the same error.

Different learners use different Algorithms (KNN, Logistic Regression), Different Hyperparameters and different training data (Cross validation, feature engineering, feature selection).

13.1. HARD VOTING

Pick the class with the most votes. There are 5 classifiers to check if an email is spam or ham. For a particular data, the prediction of the classifiers are [spam, spam, ham, ham, spam]. The final prediction of the ensemble is spam, **because 3 of the 5 models voted for spam**.

13.1.1. Hard voting with weights

There are 3 classifiers to predict class spam (1) and ham (0). The predictions from these classifiers have **weights defined as [0.1, 0.3, 0.6]**. For one email, the **predictions are [spam, spam, ham]**. For spam, we calculate the sum of weights from all classes: $\text{sum}_{\text{spam}} = w_1 * (\text{prediction}_1 == \text{spam}) + w_2 * (\text{prediction}_2 == \text{spam}) + w_3 * (\text{prediction}_3 == \text{spam}) = 0.1 * 1 + 0.3 * 1 + 0.6 * 0 = 0.4$. For ham, we calculate the same but with **ham**: $0.1 * 0 + 0.3 * 0 + 0.6 * 1 = 0.6$. The final prediction of the ensemble is ham, because the **weighted sum of ham was bigger than the sum of spam**.

13.2. SOFT VOTING

Predict the class with the **highest class probability**, averaged over all classifiers. Only possible if predictions are probabilities.

Example: There are 3 classifiers "C". For a prediction, the classifiers return the following probabilities for each class.

$C_1 = [0.85, 0.05, 0.1]$, $C_2 = [0.15, 0.15, 0.7]$, $C_3 = [0.1, 0.08, 0.82]$. The average for each class ("K") is the following:
 $K_1 = (0.85 + 0.15 + 0.1)/3 = 0.37$, $K_2 = 0.09$, $K_3 = 0.54$. Class 3 has the highest class probability and wins.

13.2.1. Soft voting with weights

There are 3 classifiers and a 3-class classification problem where we assign *equal weights* to all classifiers. The weighted average probabilities for a sample would then be calculated as follows:

Classifier	Class 1	Class 2	Class 3
Classifier 1	$w_1 * 0.2$	$w_1 * 0.5$	$w_1 * 0.3$
Classifier 2	$w_2 * 0.6$	$w_2 * 0.3$	$w_2 * 0.1$
Classifier 3	$w_3 * 0.3$	$w_3 * 0.4$	$w_3 * 0.3$

In this example, the predicted class label is 2 since it has the highest average probability.

13.3. BAGGING (BOOTSTRAP AGGREGATING)

Bagging methods form a class of algorithms which build *several models* on *random subsets* called Bootstraps of the original training set and then *aggregate their individual predictions* to form a final prediction. There are two ways of bagging: *Sampling with replacement* (Putting the cookie back in the bowl after taking it out) is called *Bagging*, *sampling without replacement* (eating the cookie after taking it out of the bowl) is called *pasting*.

Only bagging allows *data points to be used several times* for the same predictor. The individual models have a relatively *low bias and high variance*. Bagging (*reuse of data*) *reduces the variance*. This provides a way to reduce overfitting. Bagging works best with strong and complex models.

Random Subspaces: Samples are drawn as random subsets of the features. i.e. for the housing data train one model only with area and nr. of bedrooms and one with area and age of the house

Random Patches: Samples are drawn as random subsets of both samples and features.

13.3.1. Out of Bag (oob) Evaluation

If the data points are random, it is possible that some data points never get chosen. Those points are called oob-points and can be used as test-data. Since the models are only trained on a subset of data (*bootstrap* or "*in-the-bag*" data), the other data can be used as test data, the out-of-bag samples. We test every model with its oob samples and then take the majority vote / highest average probability for each oob sample to determine the accuracy with the oob Error (similar to cross validation).

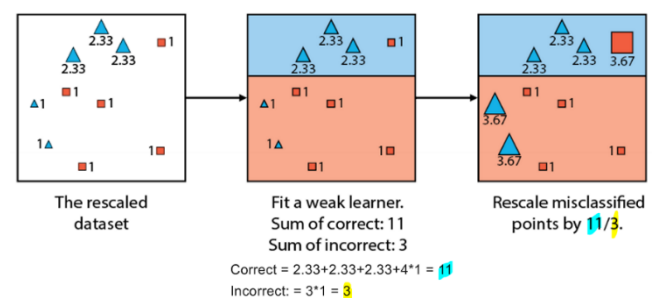
13.4. BOOSTING

Boosting is an ensemble method to train predictors *sequentially*. Each *predictor tries to correct its predecessor*. It tries to *reduce the bias* of all the combined estimators – the training error reduces.

13.4.1. AdaBoost (Adaptive Boosting)

AdaBoost assigns *equal weights* to each training sample in the beginning. Then it trains a *model to fit the given data*. After that, it *increases the weight of the misclassified samples* so they will make up a larger part of the next classifier training set, so the next classifier will perform better on them. When we stop training, we compute the weight of each predictor with *logs odds ratio*:

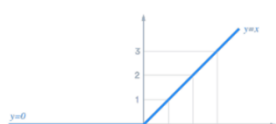
$\ln\left(\frac{\text{sum of scores of correctly classified points}}{\text{sum of scores of incorrect points}}\right)$, 0 means 50% accuracy, higher is better. To use even the bad classifiers, the results of < 0 classifiers are flipped ("do the opposite").



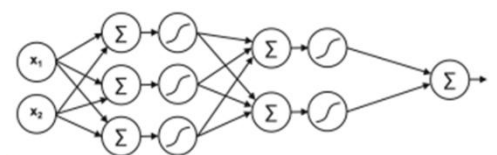
14. ARTIFICIAL NEURAL NETWORKS

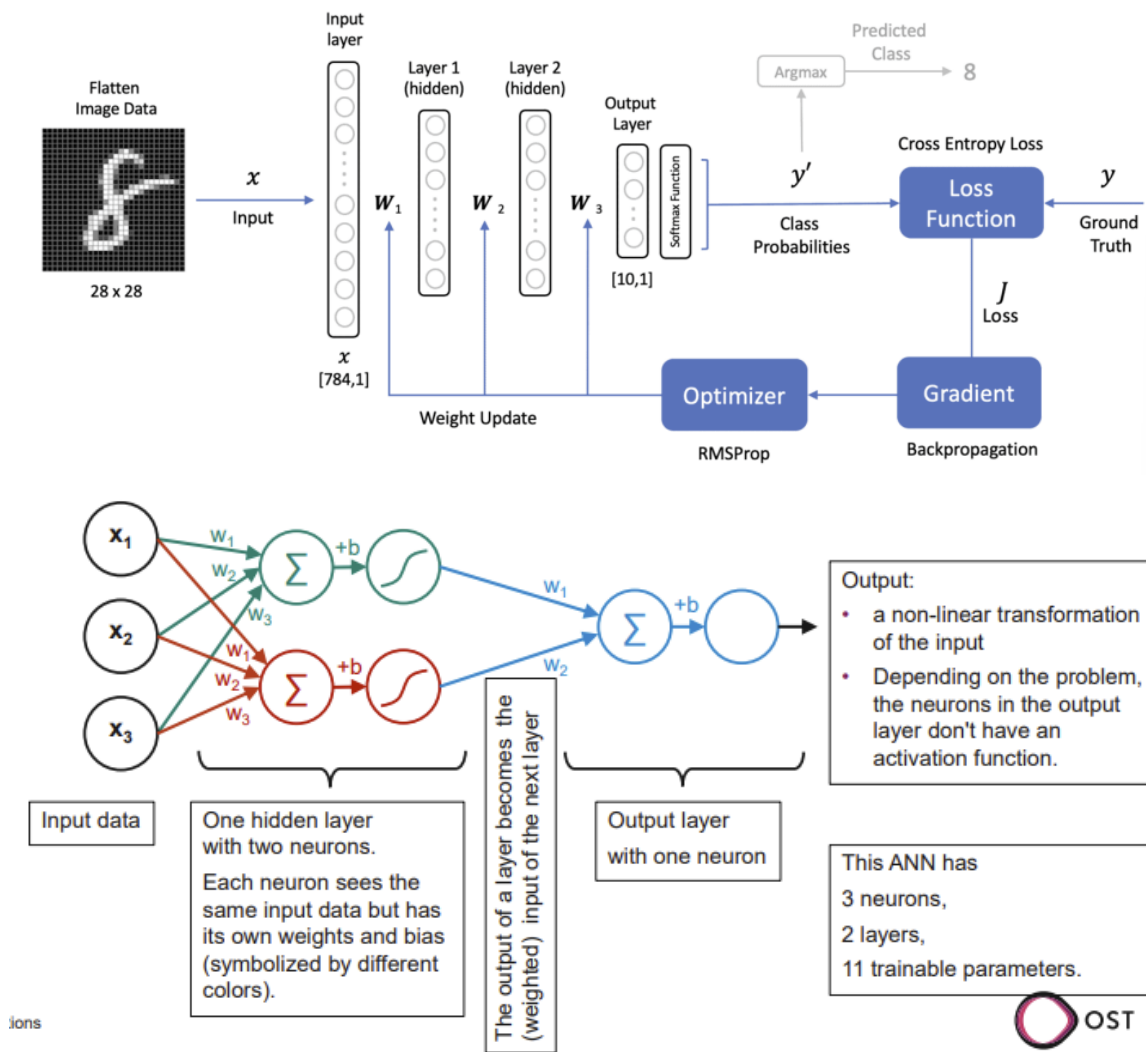
Example: 2-dimensional input, first hidden layer with 3 neurons, second hidden layer with 2 neurons, output layer with 1 neuron and without activation function. **Trainable params:** $3 * (2 \text{ weights} + 1 \text{ bias}) + 2 * (3 \text{ weights} + 1 \text{ bias}) + 1 * (2 \text{ weights} + 1 \text{ bias}) = 20 \text{ Parameter}$

ReLU Activation Function: Activation functions (make model non-linear):



- Sigmoid: Probabilities (0-1)
- SoftMax: For multiple outputs
- ReLU: For the hidden layer
- tanh: Values -1 to +1





15. LÖSUNGEN PRÜFUNGEN

15.1. HS 2021

15.1.1. Aufgabe 1: Statistik und Zufallsvariablen (Bayes)

a) Wie gross ist die Wahrscheinlichkeit, dass eine zufällig ausgewählte Person einen Blutwert zwischen 51 bis 100 hat und gesund ist, also $\Pr(X=3, Y=1)$? **0.1**

c) Wie gross ist die Wahrscheinlichkeit, dass eine zufällig ausgewählte Person gesund ist? **0.6**

d) Wie gross ist die Wahrscheinlichkeit, dass eine zufällig ausgewählte Person einen Blutwert zwischen 21 und 100 hat? **0.7**

e) Von einer Person weiss der Arzt mit Sicherheit, dass sie gesund ist. Berechnen Sie die bedingte Wahrscheinlichkeit, dass der Blutwert zwischen 51 und 100 liegt.

- Verwenden Sie Werte aus der Tabelle und die Definition $\Pr(X,Y) = \Pr(X|Y) * \Pr(Y)$.
- Notieren Sie die Formel mit den Zahlenwerten, nicht nur das Ergebnis.
- Lassen Sie ungerade Zahlen als gekürzten Bruch stehen.

$$\Pr(X=3 | Y=1) = \Pr(X=3, Y=1) / \Pr(Y=1) = 0.1 / 0.6$$

f) Berechnen Sie die bedingte Wahrscheinlichkeit, dass eine Person gesund ist, wenn der Blutwert zwischen 51 und 100 liegt.

- Verwenden Sie Bayes Rule und das Ergebnis aus der obigen Aufgabe.
- Notieren Sie die Formel mit den Zahlenwerten, nicht nur das Ergebnis.

$$\Pr(Y=1 | X=3) = \Pr(X=3, Y=1) / \Pr(X=3) = 0.1 / 0.4$$

15.1.2. Aufgabe 2: SGD und Regularisierung

Sie wollen das folgende lineare Modell auf Daten fitten: Modell: $\hat{y}_i = ax_i + b$ Das Modell wird mit SGD optimiert. Die Loss-Funktion setzt sich zusammen aus dem Mean Squared Error (MSE, mit Vorfaktor 1/2) plus einem L-1 Regularisierungsterm (aka Lasso-Regression) mit dem Regularisierungsparameter λ .

a) Notieren Sie hier die vollständige Loss-Funktion (MSE und Regularisierung)

		Blutwert 0-20	Blutwert 21-50	Blutwert 51-100	
		X=1	X=2	X=3	
gesund	Y=1	0.3	0.2	0.1	0.6
krank	Y=2	0	0.1	0.3	0.4
		0.3	0.3	0.4	1

i. Schreiben Sie die Loss-Funktion als Summe für N Datenpunkte.

ii. Schreiben Sie die Formel so aus, dass klar ersichtlich ist, wie die Daten x_i , y_i sowie die Parameter a und b in die Berechnung einfließen.

$$\frac{1}{2N} \sum_{i=1}^N (y^i - (ax^i + b))^2 + \lambda (|a| + |b|)$$

b) Zum Zeitpunkt t ist der Parametervektor $[a_t = +2, b_t = -3]$. Es wird ein Optimierungsschritt mit dem Datenpunkt $(x_i = 1, y_i = 4)$ ausgeführt. Der Regularisierungsparameter hat den Wert $\lambda = 0.1$. Berechnen Sie den Loss. Das heisst, werten die obige Formel mit den gegebenen Zahlen aus. Notieren Sie nicht nur das Endergebnis, sondern die ganze Berechnung.

$$1/2 \cdot (4 - (2 \cdot 1 - 3))^2 + 0.1 (2 + 3) = 13$$

c) Angenommen wir verwenden das obige Beispiel, aber mit $\lambda = 10^9$ (sehr gross im Vergleich zu den anderen Werten, der genaue Wert spielt keine Rolle), begründen Sie, ob die folgenden Aussagen zutreffen oder nicht. (Es ist keine Berechnung nötig um die Frage zu beantworten):

i. Das Optimum liegt praktisch bei $(a_{opt} \approx 0, b_{opt} \approx 0)$.

RICHTIG: Bei sehr grossem λ dominiert der Regularisierungsterm $\lambda(|a| + |b|)$. Das Minimum der Loss-Funktion liegt daher nahe bei $a = 0$ und $b = 0$, unabhängig vom MSE-Term.

ii. Die Parameter a und b gehen in einem einzigen Optimierungsschritt praktisch gegen 0, also $(a_{t+1} \approx 0, b_{t+1} \approx 0)$.

FALSCH: SGD macht schrittweise Updates abhängig von der Lernrate. Auch bei sehr grossem λ werden a und b nicht in einem einzigen Schritt exakt 0, sondern nähern sich über mehrere Schritte an.

15.1.3. Aufgabe 3: Lineare Regression und Cross-Validation

Gegeben sind folgende drei Datenpunkte, auf welche eine Regressionsgerade $\hat{y}_i = ax_i + b$, angepasst wird.

a) Skizzieren Sie die optimale Regressionsgerade direkt in die obige Grafik. (keine Berechnung notwendig, nur graphisch).

b) Ergänzen Sie die Residuen, beschriften Sie sie mit r_1, r_2, r_3 .

c) Direkt in der Skizze, visualisieren Sie die drei Fehlerquadrate r_i^2 .

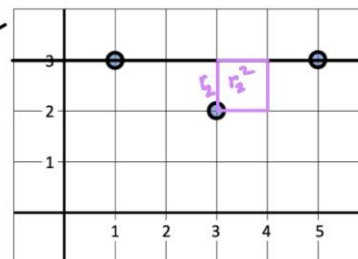
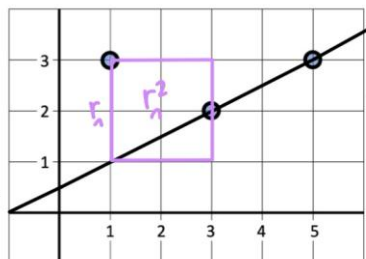
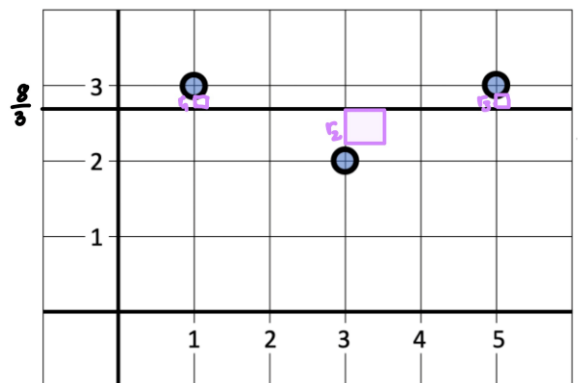
Führen Sie nun (graphisch) eine 3-fold Cross-Validation durch.

Verwenden für jeden der drei Splits je einen der drei vordruckten Graphen. In jedem Graphen:

d) Skizzieren Sie die Regressionsgerade durch die Trainings-Daten

e) Skizzieren Sie auf den Test-Daten r_1 und r_2^2

f) Aus den drei Graphen, schätzen Sie den mittleren Test-Error der durch die 3-fold Cross-Validation ermittelt wird.



15.1.4. Aufgabe 3: Model evaluation and selection

Wir haben 2 Klassifikatoren, um E-Mails in "Spam" oder "Sicher" (safe) zu trennen. Der Klassifikator gibt 1 (positive) aus, wenn die E-Mail als Spam klassifiziert wird.

a) Berechnen Sie Precision and Recall für Klassifikator #1

$$TP = 3$$

$$FP = 3$$

$$\text{recall} = TP / (TP + FN) = 3 / 3 = 1$$

$$\text{precision} = TP / (TP + FP) = 3 / (3 + 3) = 3 / 6 = 1 / 2$$

b) Berechnen Sie Precision and Recall für Klassifikator #2.

$$TP = 2$$

$$FP = 1$$

$$TN = 3$$

$$\text{recall} = TP / (TP + FN) = 2 / 2 = 1$$

$$\text{precision} = TP / (TP + FP) = 2 / (2 + 1) = 2 / 3$$

c) Ihr Spamfilter löscht automatisch die als Spam klassifizierten E-Mails. Welchen Klassifikator würden Sie für Ihre E-Mail-Klassifizierung bevorzugen? Begründen Sie Ihre Wahl. **Klassifikator 2, weil bei Klassifikator 1 potenziell wichtige Mails gelöscht werden könnten.**

Mail ID	Email Contents	Klasse/Label	Klassifikator#1	Klassifikator#2
1	Hurry Sale Tomorrow	Spam (=1)	1 TP	1 TP
2	Rain tomorrow	Spam	1 TP	1 TP
3	Sale price tomorrow	Spam	1 TP	0 FP
4	tomorrow workshop rain	Safe (=0)	1 FP	0 TN
5	tomorrow cake	Safe	1 FP	0 TN
6	Party beer	Safe	1 FP	0 TN

15.1.5. Aufgabe 4: K-NN

Wir haben den folgenden Datensatz. Jeder Datenpunkt hat zwei Features, x_1 und x_2 , sowie eine Klassenbezeichnung (labels) Spam (+) und sicher (-).

x_1	x_2	spam/safe	Platz für Berechnung Datenpunkt (-0.5, -0.5)	Platz für Berechnung Datenpunkt (1.0, 1.0)
-1	0	+	$0.5 + 0.5 = 1$	$2 + 1 = 3$
0	-1	-	$0.5 + 0.5 = 1$	$1 + 2 = 3$
1	0	-	$1.5 + 0.5 = 2$	$0 + 1 = 1$
0	1	+	$0.5 + 1.5 = 2$	$1 + 0 = 1$
1	2	+	$1.5 + 2.5 = 4$	$0 + 1 = 1$
1	3	+	$1.5 + 3.5 = 5$	$0 + 2 = 2$

a) Datenpunkt: $x_1 = -0.5$, $x_2 = -0.5$, $k = 2$, Distanz = Manhattan → kann beides sein (Spam/Safe)

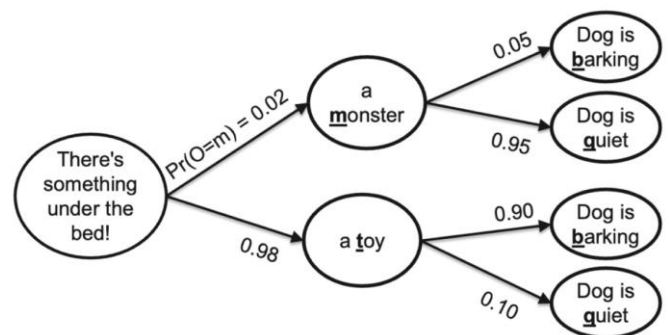
b) Datenpunkt: $x_1 = 1$, $x_2 = 1$, $k = 3$, Distanz = Manhattan → + (spam)

15.2. HS 2023

15.2.1. Aufgabe 1: Bayes Rules

a) You come home late and it's suspiciously quiet. Use Bayes' rule to calculate the probability that there is a monster under the bed given the dog is quiet. (6 Pts)

- First write the formula without inserting values, then calculate.
- Use correct notation (same pattern as used above to annotate the first arrow)
- Use the random variable O for Object. O takes the values m or t .
- Use the random variable D for Dog. D takes the values b or q .



$$P(O = m \mid b = q) = (P(b = q \mid O = m) \cdot P(O = m)) / P(D = q) = (0.95 \cdot 0.02) / (0.98 \cdot 0.1 + 0.02 \cdot 0.95) = 0.16$$

15.2.2. Aufgabe 2: Model Fitting

Scenario: Consider a dataset with $N = 5000$ samples, each sample with $m = 100$ features. A polynomial regression model of order $p = 200$ is trained on this dataset. We observe that the training error is low. The model performs poorly during validation.

a) Model fit: Overfit

b) Bias-Variance Trade-off:

The model is probably too complex, meaning it has low bias and high variance. The polynomial order $p = 200$ fits the training set very well, but small changes in the data lead to large changes in the fitted model.

→ Poor generalization on validation data.

c) Regularization choice:

L1 regularization (Lasso) is optimal because it drives some weights exactly to 0. This removes irrelevant or redundant features and reduces model complexity.

d) Cost function with regularization term:

$$\text{Loss} = \frac{1}{2N} \sum_{j=1}^N (y_j - \hat{y}_j)^2 + \lambda \cdot \sum_{j=1}^p |w_j|$$

15.2.3. Aufgabe 3: Working with Data in Python

In the class exercises, we used a powerful library to read, analyze and manipulate data. Look at the following line of Python code. It reads a csv file. $df = pd.read_csv('myfile.csv')$

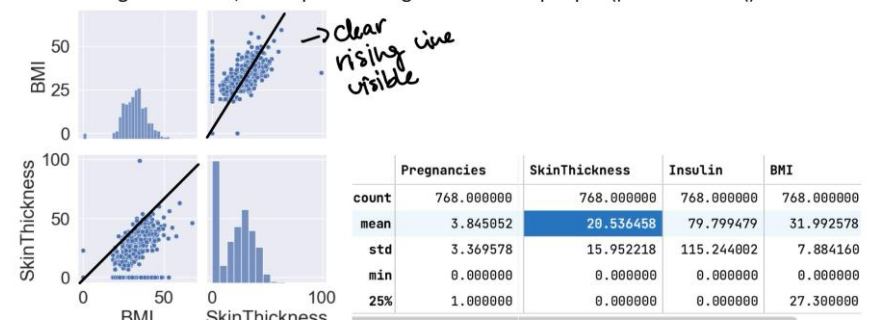
a) Which Python package is used here to read the csv file? **Pandas**

b) What is the datatype of the object returned by `read_csv`? **Pandas DataFrame**

c) A data analyst says: "BMI and skin thickness are correlated". Explain the statement and annotate the figure to visualize what the data analyst is saying. No calculation needed. This means that if the BMI increases, the skin thickness also increases.

The cloud of points forms a clear upward-sloping pattern.

After loading the dataset, it is explored using the functions `pairplot()` and `describe()`:



d) The `describe()` function calculates a mean "SkinThickness" of ~20.53. The data analyst tells you: "There is something wrong with the dataset. The mean "SkinThickness" should be around 30". Explain why the data analyst is right. According to the table, around 25% of the values are 0, which indicates that we might have many missing values that were encoded as 0. These zeros pull the mean down, which explains why the average SkinThickness is around 20 instead of the expected value around 30.

15.2.4. Aufgabe 4: Word Representations and Cosine Similarity

Write the formula for the Cosine-Similarity for the simplified case where all vectors are normalized (length of each vector = 1). $\cos(\Theta) = x \cdot y$. What does the symbol Θ in the above formula refer to? The angle between the two vectors x and y . It measures how similar the directions of the two vectors are.

In terms of cosine similarity, is `word_3` more similar to `word_1` or to `word_2`? Do a calculation to justify your answer. The closer to 1 the more similar they are. Dot product of 1 & 3: $0.284 \cdot 0.047 + 0.946 \cdot 0.977 + 0.158 \cdot -0.209 = 0.904$

15.2.5. Aufgabe 5: Adaboost

The above figure shows the decision boundary (line $x = 2.0$) after the first iteration of AdaBoost. We address the classifier after the 1st iteration as weak learner 1 (WL1). The WL1 predicts points to the left of the boundary as positive and those to the right as negative. Answer the following questions based on the above information.

a) Draw the confusion matrix for WL1. Calculate accuracy and precision for WL1.

$$\text{accuracy} = (TP + TN) / \text{total} = (10 + 5) / 23$$

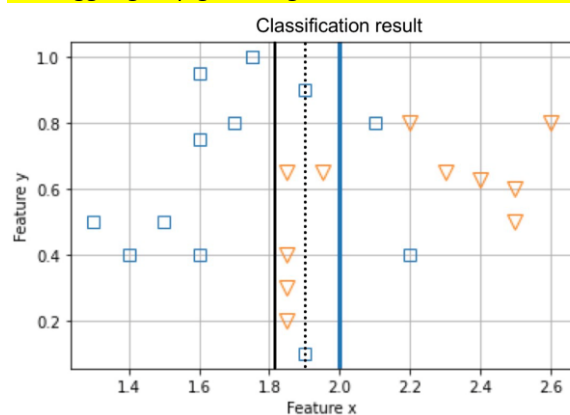
$$\text{precision} = TP / (TP + FP) = 10 / (10 + 5) = 10/15$$

	$\wedge^+$	$\wedge^-$
+	10	2
-	5	6

b) In the next iteration, Adaboost assigns new instance weights. In the graph below, highlight the points with higher instance weights using circles.

c) Draw a solid line that roughly shows the decision boundary for WL2 (weak learner after 2nd iteration). State your reasoning.

The bigger group gets weighted more and thus it shifts to the left



f) Adaboost is terminated after completing the 2nd iteration. An ensemble is created. Roughly draw a dashed line in the above plot as the ensemble's decision boundary based on the boundaries of WL1 and WL2. State your reasoning. The final ensemble is a decision between the two previous points

15.3. FS 2024

15.3.1. Aufgabe 1: Bayes

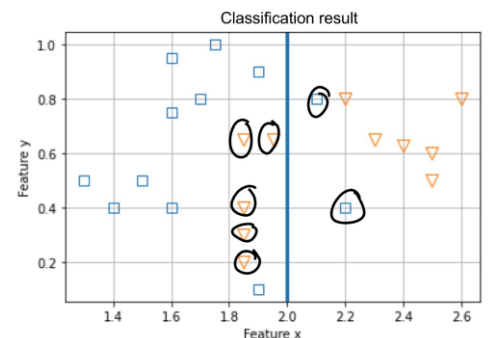
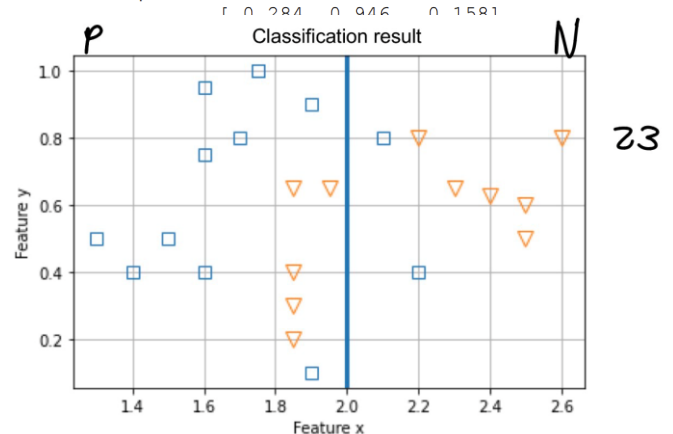
A bot performs some task on a computer. An Intrusion Detection Systems (IDS) is running to detect malware by analyzing internet traffic. Malware generates a different outbound traffic pattern than the bot. The following table shows the joint probabilities. Assume that either the bot or the malware is active, not both at the same time.

a) Calculate all marginal probabilities. Write the result directly into the table above.

Now consider the following Python script and its output:

```
1 import numpy as np
2
3 word_1 = np.array([ 0.9, 3, 0.5])
4 word_1 = word_1 / np.linalg.norm(word_1)
5 word_1 = np.round(word_1,3)
6 print(word_1)
7
8 word_2 = np.array([-2, 3.3, -0.5])
9 word_2 = np.round(word_2 / np.linalg.norm(word_2), 3)
10 print(word_2)
11
12 word_3 = np.array([0.2, 4.2, -0.9])
13 word_3 = np.round(word_3 / np.linalg.norm(word_3), 3)
14 print(word_3)
15
16 dot_product12 = np.dot(word_1, word_2)
17 print("dot_product word_1, word_2 = {}".format(dot_product12))
18 dot_product23 = np.dot(word_2, word_3)
19 print("dot_product word_2, word_3 = {}".format(dot_product23))
```

Output:



$\text{Pr}(U,T)$

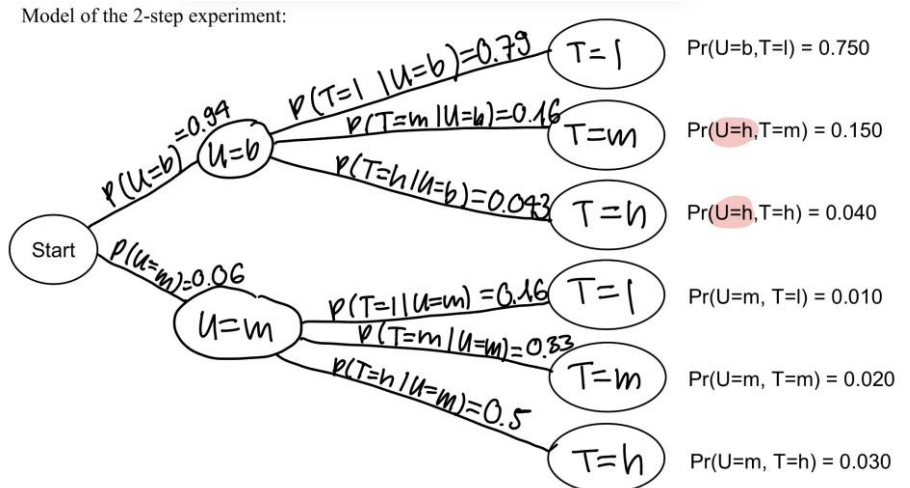
		outbound traffic (T)			
		T=low	T=medium	T=high	
user (U)	U=bot	0.750	0.150	0.040	0.94
	U=malware	0.010	0.020	0.030	0.06
		0.76	0.17	0.07	1

b) A colleague makes the following statement: "High traffic is a signature of the bot, not of the malware because the table shows 0.030 vs. 0.040.". Is he right or wrong? Explain! **Wrong: The table only shows joint probabilities, but the question asks for a conditional probability. The conditional probability also considers that bots are more frequent than malware.**

c) Model the outbound traffic as a two-step experiment. Complete the graph on the next page with the missing nodes and arrows. d) Inside each node write the event it represents. For example "T=h". e) Along each arrow, write an expression of the form $\Pr(\dots)=\dots$

f) Use Bayes' Rule to answer the question: "The Intrusion Detection Systems (IDS) observes high traffic. What is the probability, that a malware is active?"

$$\Pr(U=m \mid T=h) = \Pr(U=m, T=h) / \Pr(T=h) = 0.03 / 0.07$$



15.3.2. Aufgabe 2: Regularization

You want to predict a target variable y for some input feature x . The underlying unknown input function (generated from a polynomial of order 5) is partially shown by the gray line. Some of the observed noisy data points are shown in blue. The target prediction of the model is written as $\hat{y}$. You now train a polynomial regression model of order M for prediction using the following code snippet.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, shuffle=True)
poly = PolynomialFeatures(M, include_bias = False)
poly.fit_transform(X_train)
linear_regression = LinearRegression(fit_intercept = True)
linear_regression.fit(X_train, Y_train)
```

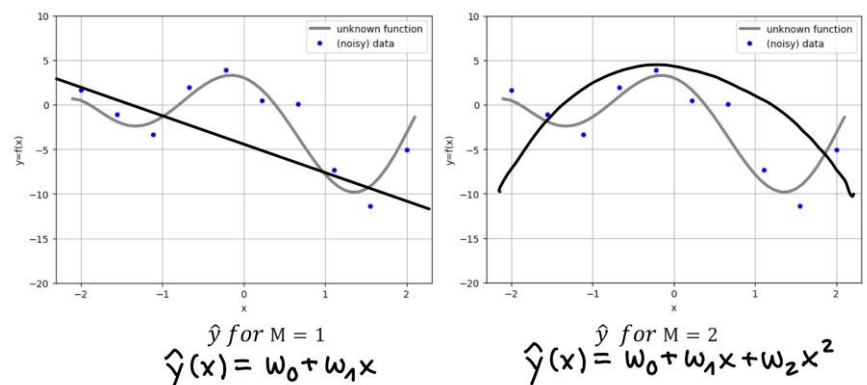
→ creates features $x, x^2, x^3, \dots$
 adds w_0
 apply linreg must be linear, that's why we use weights

a) Write the regression model formula corresponding to the above code. $\hat{y} = w_0 + \sum_{i=1}^M w_i x^i$

b) Write the error/loss function that your model tries to minimize. $E = \frac{1}{2N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$

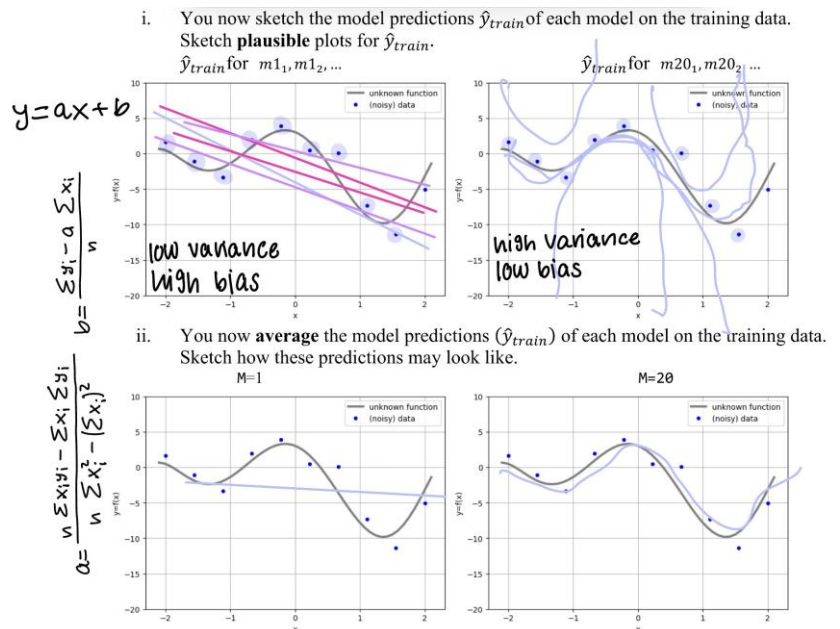
c) Sketch plausible plots for $\hat{y}$ for $M=1$ and $\hat{y}$ for $M=2$.

d) When asked for the best value of M , you choose the value of M to be 50. You argue that the training error and test error reduce monotonically as we increase M . Discuss. If M is large, we get an overfitting which means that we get a high variance and thus the test error increases. The training error reduces, and we get a near perfect fit. Low training error != good model.



e) You now choose a simple model with $M=1$ and a more complex model with $M=20$. You run the model training pipeline 5 times. This results in 5 trained models, each for $M=1$ and $M=20$. Let us call these models as $m1^1, m1^2, \dots$ for $M=1$ and $m20^1, m20^2, \dots$ for $M=20$. Hint: Notice that `train_test_split` shuffles the dataset and returns a different split each time – you train on a different split each time.

iii.) Explain the effect of model complexity M on bias and variance using your plots. Increasing model complexity reduces bias but increases variance. For $M=1$ the model underfits with high bias and low variance. For $M=20$ the model fits the data well but shows high variance. Averaging predictions reduces variance but does not reduce bias.



15.3.3. Aufgabe 3: Gradient Descent Optimization

We optimize a two-parameter model using Gradient Descent. The model and the dataset are not relevant. The MSE is used. The graph shows the "Loss Landscape", with black gridlines and contours overlayed for readability. Example Vectors are added to provide the scale. Solve the question using the graph; no calculations required.

a) Estimate the best possible parameters. Try to be as accurate as ± 0.5 units for the params, and ± 10 units for the Loss: $p1_{best} = 5$ $p2_{best} = -4.5$ $Loss_{best} = 10$

b) Graphically perform 2 update steps: 1. The optimizer is initialized with $[p1 = -2, p2 = -6]$, mark this with a cross and write "A" next to it. 2. Perform a Gradient Descent step in the given error landscape. Assume that the learning rate is such that the step-size is 2cm. Draw an arrow indicating this update step. 3. Perform a second update step. Assume the step-size is 1cm.

c) The optimizer is now reinitialized with $[p1 = +6, p2 = -6]$. Graphically perform one update step 1. Mark the start location with a cross and write "B" next to it. 2. Perform one Gradient Descent step. Assume the step-size is 4cm.

3. How does the Loss change? Explain the problem and how it could be solved. The learning rate (step size) is too large. It causes overshooting. Fixes: Reduce learning rate

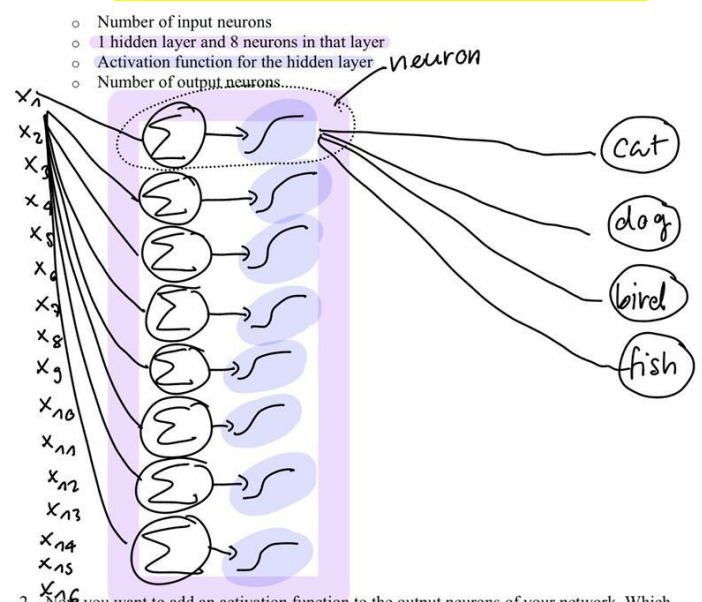
15.3.4. Aufgabe 4: Neural Networks

You are working with a neural network that classifies images of animals into one of four classes: cat, dog, bird, or fish. You are tasked with designing a simple feedforward neural network to classify grayscale images of size 4×4 pixels into these four classes.

1. Specify and draw your network showing: 2. Now you want to add an activation function to the output neurons of your network. Which activation function will you use? Explain why it is a suitable choice for this task. Softmax, because this is a 4-class classification problem and the outputs should form a probability distribution.

3. Calculate the total number of trainable parameters in your network, considering weights and biases. Show all the steps of your calculation. 1. Hidden layer: $8 \cdot (16+1)$, Output: $4 \cdot (8+1) \rightarrow$ total = 172 $\rightarrow$ formula: sum of all hidden layers + output (neurons $\cdot$ (weights + 1 bias))

4. Can adding polynomial features to logistic regression make it as expressive as a neural network with a single hidden layer? Justify your answer. Points will be awarded only if a justification is provided. Adding polynomial features increases the expressiveness of logistic regression and allows non-linear decision boundaries. However, it is still limited to polynomial functions with a fixed feature expansion. A neural network with one hidden layer is more flexible and, according to the universal approximation theorem, can approximate arbitrary functions. Therefore, polynomial logistic regression is not as expressive as a neural network.



15.4. THEORY QUESTION 2021 PART 1

15.4.1. PMF, Gleichverteilte Zufallsvariable (uniform probability mass function)

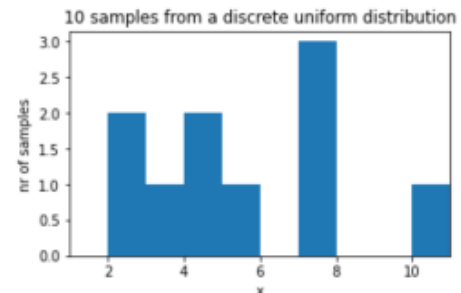
Angenommen Sie verwenden eine Uniform-Distribution um 10 ganzzahlige Zufallszahlen zwischen 1 und 10 zu erzeugen

- Skizzieren Sie ein qualitatives, plausibles Histogramm mit 10 Bins.
- Beschriften Sie den Graphen

-Ein Histogramm in welchem jeder Wert von 1 bis 10 genau 1 mal vorkommt wäre falsch (da extrem unwahrscheinlich).

-korrekte Beschriftungen. Titel, x-label, y-label. Die Position der x-ticks kann variieren, Pyplot macht sie am linken Rand des Balkens. Y-ticks wären besser Ganzzahlig

array([5, 7, 10, 4, 4, 2, 3, 7, 7, 2])

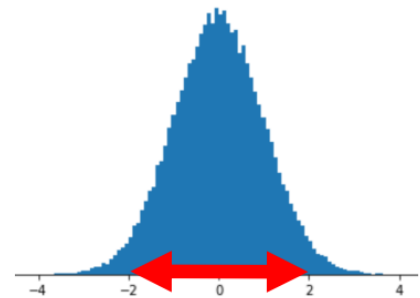


15.4.2. Normalverteilte Zufall

Die folgende Abbildung zeigt ein Histogramm. Die Daten wurde folgendermassen erzeugt: `rng.normal(loc=0, scale=1, size=10000)`

- Ungefähr wie viele Samples fallen in den rot markierten Bereich $[-2, +2]$

Die gegebene Normalverteilung hat den Mittelwert 0 und die Standardabweichung 1. Das Intervall $[-2, +2]$ ist somit um den Mittelwert zentriert, von -2 Standardabweichungen bis +2 Standardabweichungen. In diesem Intervall liegen rund 95% der Samples, also rund 9500



15.4.3. Wahrscheinlichkeiten, joint- Marginal-, Conditional- Probability

Suppose you receive Emails with text described in the following table. The table also shows if the emails are spam or not

No	Email Text	Spam
1	Hurry Sale Tomorrow	1
2	Rain tomorrow	0
3	Sale price tomorrow	1
4	tomorrow workshop rain	0

Calculate the following:

- The probability that any message in the above dataset is spam

$$\Pr(\text{spam}) = 0.5$$

2 von 4 Mails sind Spam $2/4=0.5$

- The probability that any message in the above dataset is not-spam

$$\Pr(\text{not-spam}) = 1 - 0.5 = 0.5$$

Ebenfalls 2/4 Man kann aber auch die "Gegenwahrscheinlichkeit" berechnen: $\Pr(\text{not-spam}) = 1 - \Pr(\text{spam}) = 1 - 0.5$

- Calculate the conditional Probabilities in the probability table below

Meaning of notation:

$\Pr(\text{Hurry} | \text{spam})$ = Given that the email is spam, what is the probability that the email contains "Hurry"?

$\Pr(\text{Hurry} | \text{not-spam})$ = Given that the email is not-spam, what is the probability that the email contains "Hurry"?

Beispiel: $\Pr(\text{Hurry} | \text{spam})$:

Es gibt zwei Spam Mails. Hurry kommt in einem dieser Spam Mails vor, deshalb: $\Pr(\text{Hurry} | \text{spam}) = 1/2 = 0.5$

Word	$\Pr(\text{word} \text{spam})$	$\Pr(\text{word} \text{not-spam})$
Hurry	$1/2 = 0.5$	$0/2 = 0$
Sale	$2/2 = 1$	$0/2 = 0$
Tomorrow	$2/2 = 1$	$2/2 = 1$
Rain	$0/2 = 0$	$2/2 = 1$
Workshop	$0/2 = 0$	$1/2 = 0.5$
Price	$1/2 = 0.5$	$0/2 = 0$

Hint: How often does the word occur in spam or not-spam emails?

d) Calculate the following probabilities:

Pr("Hurry"): what is the probability that an email contains "Hurry"?

Word	Pr(word)
Hurry	$1/4 = 0.25$
Sale	$2/4 = 0.5$
Tomorrow	$4/4 = 1$
Rain	$2/4 = 0.5$
Workshop	$1/4 = 0.25$
Price	$1/4 = 0.25$

e) Calculate the conditional probabilities in the following tables

Pr(spam | "Hurry"): Probability that an email containing "Hurry" is a spam

Word	P(spam word)	P(not-spam word)
Hurry	$\frac{\text{Pr('hurry' spam)} \text{Pr(spam)}}{\text{Pr('hurry')}} = \frac{0.5 * 0.5}{0.25} = 1$	0
Sale	$\frac{1 * 0.5}{0.5} = 1$	0
Tomorrow	$\frac{1 * 0.5}{1} = 0.5$	0.5
Rain	$\frac{0 * 0.5}{0.5} = 0$	1
Workshop	$\frac{0 * 0.5}{0.25} = 0$	1
Price	$\frac{0.5 * 0.5}{0.25} = 1$	0

Hint: Bayes Theorem

$$\text{Pr}(spam|word) = \frac{\text{Pr}(word|spam) \text{Pr}(spam)}{\text{Pr}(word)}$$

15.4.4. Lineare und Polynomiale Regression. Bias-variance trade-off

Sie bekommen folgende Daten und fitten ein kubisches Modell an diese Daten:

- a) Formulieren Sie das Modell als mathematische Gleichung $\hat{y} =$

$$\hat{y}_i = w_3 x_i^3 + w_2 x_i^2 + w_1 x_i + w_0$$

$$e_i = y_i - \hat{y}_i$$

$$\text{MSE} = (1 / 2N) \sum e_i^2$$

$$\text{MSE} = (1 / 2N) \sum (y_i - (w_3 x_i^3 + w_2 x_i^2 + w_1 x_i + w_0))^2$$

- b) Wie viele freie Parameter hat das Modell?

Das Modell hat 4 Parameter

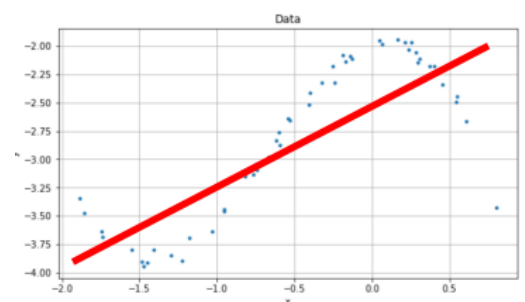
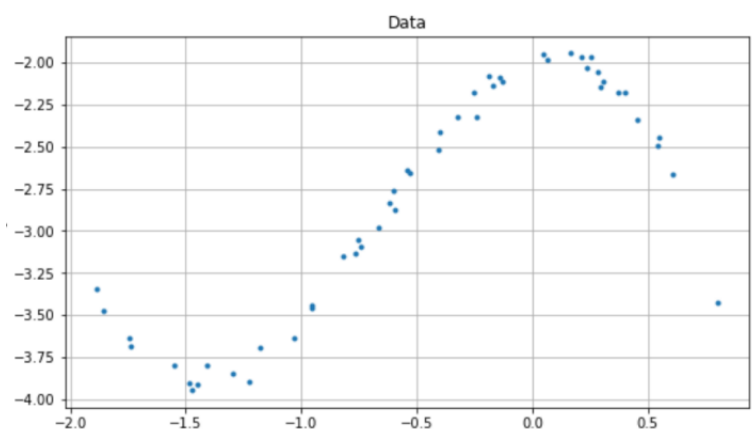
- c) Formulieren Sie den MSE als mathematische Gleichung. MSE = ... (siehe oben bei a))

-Anstelle von w kann natürlich auch beta, theta, abcd, etc. Verwendet werden.

-streng genommen ist der MSE nicht durch 2 geteilt. Es wird praktisch aber so gemacht, da sich beim Ableiten (für Optimierung mit SGD) die 1/2 mit der 2 aus (...)^2 kürzt

-Das Modell kann mit oder ohne den Subskript i geschrieben werden. Beides OK. Beim MSE braucht es aber einen Index (z.B. i) und die Summe muss von 1 bis N gehen.

-Die Parameter können auch anders nummeriert werden, gemischte Indices wie $w_1 x^3$ sind aber eher verwirrend



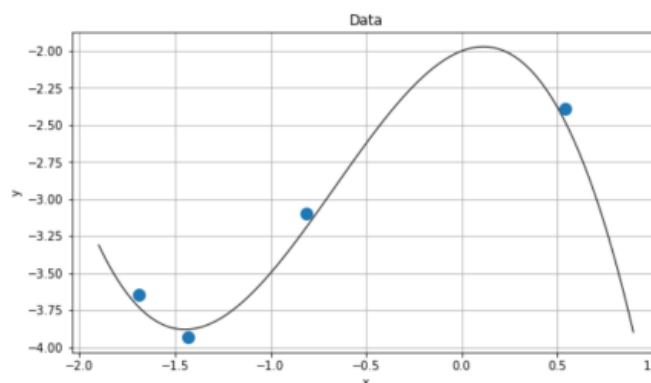
Nehmen Sie an, jemand würde ein einfaches lineares Modell (eine Gerade) an die oben gegebenen Daten fitten

- d) Skizzieren Sie das Ergebnis (die Gerade) direkt in die oben gegebene Figur.
e) Hat der Generalisierungsfehler des linearen Modells einen grossen oder kleinen Bias?
Das ist ein Beispiel für einen hohen/grossen Bias. (und tiefe Variance, underfitting).

- f) Nehmen Sie an, sie würden untenstehende vier Datenpunkte erhalten.

Ist es eine gute Idee, ein Polynom 3. Ordnung zu fitten?

Nein. Das kubische Modell hat vier Parameter. Wenn man mit nur vier Datenpunkten vier freie Parameter bestimmen will, findet man ein Polynom, welches exakt durch die Datenpunkte geht (der Training-Error wird 0, Overfitting). Da aber die Daten mutmasslich (in der Praxis immer) fehlerbehaftet sind (z.B. auch nur kleine Messfehler), muss man erwarten, dass ein «exakt auf die Fehler» gefittetes Modell schlecht generalisiert



15.4.5. SGD

Ein Modell mit zwei Parametern (a und b) wird auf Daten gefittet. Das Modell wird mit SGD optimiert.

Nach der Iteration t befindet sich der Optimizer an der Stelle (a=0.6, b=0.4).

An dieser Stelle werden die folgenden partiellen Ableitungen des Losses 'E' ausgewertet:

$$\partial E / \partial a = +0.20$$

$$\partial E / \partial b = -0.30$$

Nehmen Sie $\alpha = 0.1$ an.

- a) Berechnen Sie den Parametervektor zum Zeitpunkt t+1.

$$[a, b]_{t+1} = [a, b]_t - \alpha \cdot [\partial E / \partial a, \partial E / \partial b]$$

$$(0.6, 0.4) - 0.1 \cdot (0.2, -0.3) = (0.58, 0.43)$$

15.4.6. Logistic Regression

Suppose the task is to predict the admission decision of an applicant given his secondary school score (x1) and BS grades (x2).

Given some data and using logistic regression, we train a model that has the following parameters

$$\Pr(\text{accept} \mid x; w) = 1 / (1 + e^{-(w_1 \cdot x_1 + w_2 \cdot x_2 + w_3)})$$

$$w_1 = 1$$

$$w_2 = -0.7$$

$$w_3 = -0.7$$

- a) Calculate the probability that an applicant get i) rejected and ii) accepted if Secondary school score = 4.5 and BS grad = 5.5

```
1 linear_score = lambda x1, x2: x1-0.7*x2-0.7
```

```
1 linear_score(4.5,5.5)
```

```
-0.04999999999999996
```

```
1 np.exp(-linear_score(4.5,5.5))
```

```
1.0512710963760237
```

```
1 prob_accept = 1/(1+np.exp(-linear_score(4.5,5.5)))
```

```
1 prob_reject = 1-prob_accept
```

```
1 print(prob_accept)
```

```
2 print(prob_reject)
```

```
0.4875026035157898
```

```
0.5124973964842102
```

- b) Write down the formula for the Loss function used in logistic regression. Assume N data points

$$-\frac{1}{N} \sum_{i=1}^N y_i \log(p_i) + (1 - y_i) \log(1 - p_i)$$

- c) Write the Loss (the formula, without calculating a value) for the single data point given above. You can plug-in the probability you found in a). Assume y=0 for rejected and y=1 for accepted (see below)

d) Calculate the Loss for the two possibilities: i) if the applicant was rejected ($y=0$) and ii) if the applicant was accepted ($y=1$)

$$-\frac{1}{N} \sum_{i=1}^N y_i \log(p_i) + (1 - y_i) \log(1 - p_i)$$

For $N=1$, and plugging in the probabilities found above, we have:

$$-1 \cdot (y_i \cdot \log(0.4875) + (1 - y_i) \log(0.5125))$$

If the participant was rejected:

That means: what is the contribution of the data point ($x_i = (4.5, 5.5)$, $y_i = 0$) to the total loss?

$$\begin{aligned} \text{loss} &= -1 \cdot (0 + (1 - 0) \log(0.5125)) \\ &= 0.6685 \end{aligned}$$

If the participant was accepted:

That means: what is the contribution of the data point ($x_i = (4.5, 5.5)$, $y_i = 1$) to the total loss?

$$\begin{aligned} \text{loss} &= -1 \cdot (1 \cdot \log(0.4875) + 0) \\ &= 0.7185 \end{aligned}$$

Note that the loss is composed of two complementary terms. For each data point, only one of the two is evaluated, because either y_i or $1 - y_i$ equals 0.

Note that the loss is larger if the participant was accepted. This reflects well the fact that according to the model the participant has a (slightly) higher probability of being rejected.

15.5. THEORY QUESTION 2021 PART 2

15.5.1. Confusion Matrix für binäre Klassifikation

Ein Classifier wird mit $N=100$ Datenpunkten trainiert. Gegeben sind TP, FN und FP. Berechnen Sie die gesuchten Kennzahlen:

TP = 40 FN = 10

FP = 20 **TN = 30**

a) Anzahl Datenpunkte mit Label «positiv» = **40+10=50**

b) Anzahl Datenpunkte mit Label «negativ» = **20+30=50**

c) Mean Accuracy = **$(TP+TN)/N = (40+30)/100 = 0.7$**

d) Mean Error = **$(FP+FN)/N = (20+10)/100 = 0.3$**

e) Precision = **$TP/(TP+FP) = 40/(40+20) = 4/6 = 2/3 \sim 0.66$**

f) Sensivity (=TPR) = **$TP/(TP+FN) = 40/(40+10) = 4/5 = 0.8$**

g) Miss Rate (=FNR) = **$1 - \text{TPR} = 0.2$**

h) False Positive Rate = **$FP/(FP+TN) = 20/(20+30) = 2/5 = 0.4$**

i) Lässt sich die Qualität dieses Classifier in einer ROC-Kurve darstellen?

Man kann (FÜR=0.4, TPR=0.8) im ROC-Raum darstellen. Das gibt dann **einen einzigen Punkt**, keine Kurve. Für eine Kurve müsste man den Classifier mit allen (vielen) Schwellenwerten (Threshold) im Intervall $[0, 1]$ auswerten

10) KNN

Betrachten Sie nachfolgende Daten. Sie werden mit KNN ($N=12$, Manhattan-Distanz) klassifiziert

a) Klassifizieren Sie den neuen Datenpunkt x

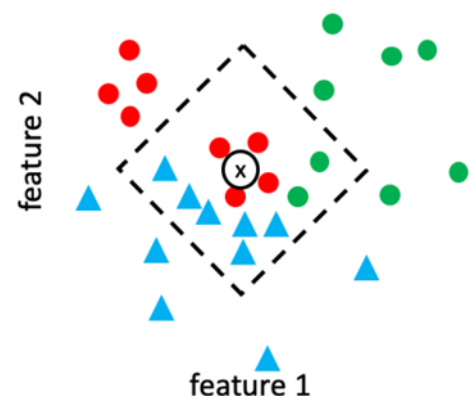
[]rot []grün [x]blau (6 neighbours)

b) Ergänzen Sie (ungefähr) die Abbildung so, dass klar wird, wie Sie zu ihrem Resultat kommen.

-Das stehende Quadrat zeigt Punkte, welche dieselbe Manhattan-Distanz vom Punkt x haben

-Das Quadrat muss so gross sein, bis es genau $N=12$ Neighbours einschliesst

-Es wäre falsch einen Kreis zu skizzieren



Cross Validation und Regularisierung

Ein Machine-Learning Problem mit zwei unterschiedlichen Werten für den Regularisierungsparameter optimiert und mit Cross-Validation evaluiert. Die Ergebnisse stehen in folgender Tabelle. Analysieren Sie die Werte und beantworten Sie die Fragen

$\lambda=0.005$	Fold	performance score (higher = better)		
	1	95		
	2	30		
	3	70	sum	195
			avg	65
$\lambda=0.01$	Fold	performance score (higher = better)		
	1	80		
	2	75		
	3	65	sum	220
			avg	73.3

Welche der Antworten sind richtig (mehrere Antworten möglich):

- ☐ Man sollte das Modell mit dem Performance Score 95 verwenden
- ☒ Man sollte ein neues Modell trainieren
- ☒ Der Regularisierungsparameter 0.01 liefert das bessere Ergebnis
- ☒ Man sollte ein neues Modell auf allen Trainingsdaten mit $\lambda=0.01$ trainieren

Begründung:

Mit CV kann man die Generalisierungseigenschaften eines Modells abschätzen. Der durchschnittliche score ist im zweiten Fall besser $220/3 > 195/3$. Das bedeutet, dass eine stärkere Regularisierung ($\lambda=0.01$) im Durchschnitt bessere Ergebnisse liefert. Man würde nun ein neues Modell auf allen Trainingsdaten, mit $\lambda=0.01$ trainieren. Man kann noch ergänzen, dass $\lambda=0.01$ zu einem stärkeren Bias führt, dass aber die Varianz tiefer ist.

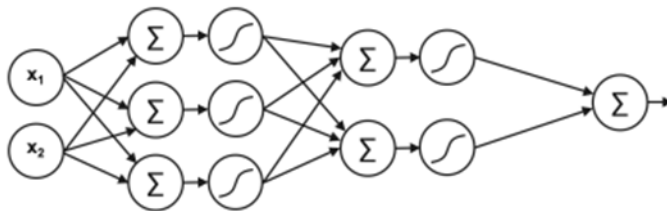
ANN

a) Skizzieren Sie ein ANN gemäß folgenden Angaben. Verwenden Sie die Darstellung aus der Vorlesung

- zweidimensionales Input
- erster hidden Layer mit drei Neuronen
- zweiter hidden Layer mit einem Neuron und ohne Aktivierungsfunktion

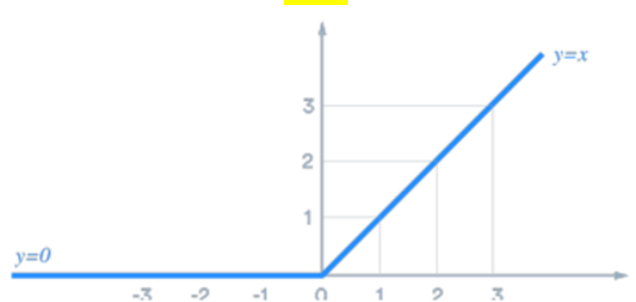
b) Wieviele «trainable params» hat dieses Netz?

c) Skizzieren Sie den Funktionsgraphen der ReLU – Aktivierungsfunktion



erster hidden Layer: $3 \cdot (2 \text{ weights} + 1 \text{ bias}) = 9 \text{ parameter}$
 zweiter hidden Layer: $2 \cdot (3 \text{ weights} + 1 \text{ bias}) = 8 \text{ parameter}$
 output Layer: $1 \cdot (2 \text{ weights} + 1 \text{ bias}) = 3 \text{ parameter}$
 total: 20 Parameter

ReLU:



16. LÖSUNGEN ÜBUNGEN

16.1. ÜBUNG 2:

Assume an algorithm (for example word2vec) has found the following representations for the words "apple", "orange" and "bike":

Before calculating a value, guess the cosine similarity $SC(\text{"apple"}, \text{"orange"})$ and $SC(\text{"apple"}, \text{"bike"})$. apple and orange are similar -> SC close to 1. apple and bike are different -> SC is smaller but not 0

apple	orange	bike
$\begin{bmatrix} 3 \\ 6 \\ 2 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 7 \\ 2 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 0 \\ 8 \\ 7 \end{bmatrix}$

Calculate the cosine similarity $S_c(\text{"apple"}, \text{"orange"})$ and $S_c(\text{"apple"}, \text{"bike"})$.

```
# Define vectors
apple = np.array([3, 6, 2, 1])
orange = np.array([2, 7, 2, 0])
bike = np.array([1, 0, 8, 7])
new_word = np.array([0, 0.9, 12, 7])

# Calculate the length of the vector
length_apple = np.linalg.norm(apple)
print("length_apple={}".format(length_apple))

length_orange = np.linalg.norm(orange)
print("length_orange={}".format(length_orange))

dot_product = np.dot(apple, orange)
print("dot_product={}".format(dot_product))
cosine_similarity = dot_product / (length_apple * length_orange)
print("cosine_similarity={}".format(cosine_similarity))

length_apple=7.0710678118654755
length_orange=7.54983443527075
dot_product=52
cosine_similarity=0.9740492440449616
```

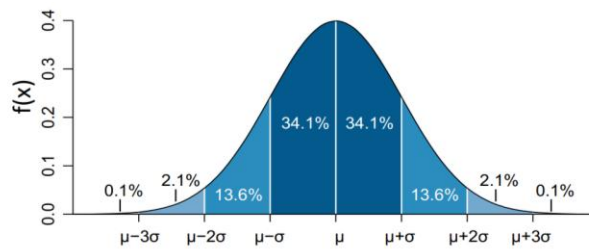
Why does it make sense to normalize the vector representations? How would the calculation simplify (become more efficient) if vectors are normalized, the denominator becomes $1 \times 1 = 1$. Calculating the cosine similarity reduces to the dot product. You normalize a vector once, in a preprocessing-step, later, you save computations each time you use that normalized vector in a cosine similarity calculation.

Let's assume that the representation was developed to classify words into fruits and

vehicles. Do you think the following vector represents a fruit or a vehicle? Calculate the cosine similarities. If the new word is similar to apple (and orange), it's likely a fruit. If it's more similar to bike, the new word will likely be classified a vehicle. From the calculations below, we see that the new vector is closer to the bike-vector than to the apple vector, and therefore probably represents a vehicle.

16.2. ÜBUNG 3

The following graph shows the PDF of a gaussian distribution with mean μ and standard-deviation σ . Theoretically, a gaussian random-variable has a range from $-\infty$ to $+\infty$, but the probability to take a value far from the mean goes quickly to very small values (practically 0).



https://commons.wikimedia.org/wiki/File:Standard_deviation_diagram_micro.svg

For continuous random variables, we cannot read the probability of events directly from the probability density function; the values on the y-axis are **not** probabilities! Instead, the "area under the curve" (= the integral) corresponds to the likelihood of an event to fall into an interval $[a, b]$:

$$Pr(a < X < b) = \int_a^b f(x)dx$$

- a) A random variable X follows a gaussian distribution. What is the probability for X to take a value in the interval $[\mu - 2\sigma, \mu + 2\sigma]$

Formally, we are interested in the area under the curve in the given interval:

$$Pr(\mu - 2\sigma < X < \mu + 2\sigma) = \int_{\mu - 2\sigma}^{\mu + 2\sigma} f(x)dx$$

We can simply approximate that area using the values in the figure above:

$$13.6 + 34.1 + 34.1 + 13.6 \approx 95.4\%$$

- b) A Gaussian random variable Y has mean $\mu = 45$ and $\sigma = 10$.

- i) What is the probability that Y takes a value < 45

By symmetry, this is 50%

- ii) What is the probability that Y takes a value between 35 and 65 ?

You should note that this interval can be expressed in terms of mean μ and std-dev σ :

$$Pr(\mu - 1\sigma < Y < \mu + 2\sigma) \approx 34.1 + 34.1 + 13.6 \approx 81.8\%$$

- iii) What is the probability that Y takes the (exact) value 45?

$$\int_{x=45}^{x=45} f(y)dy = 0$$

The area under the "curve" is 0!

x_1	x_2	y
1	2	2
2	3	3
3	4	5

16.3. ÜBUNG 8: REGULARIZATION

Consider the linear regression model $y = w_1 * x_1 + w_2 * x_2$, where w_2 and w_1 are the weights (coefficients), x_1 and x_2 are the features, and y is the target variable. Let's say we have the following dataset: And we have initial weights $w_1 = 0.5$ and $w_2 = 0.5$.

16.3.1. MSE

Calculate the Mean Squared Error (MSE) loss. The formula for MSE is:

$$MSE = 1/N * \sum (y_i - (w_1 * x_{1i} + w_2 * x_{2i}))^2$$

Where y_i is the actual output, $(w_1 * x_{1i} + w_2 * x_{2i})$ is the predicted output ($\hat{y}$), and N is the number of data points.

First, calculate the predicted output for each data point:

$$\hat{y}_1 = 0.5 * 1 + 0.5 * 2 = 1.5$$

$$\hat{y}_2 = 0.5 * 2 + 0.5 * 3 = 2.5$$

$$\hat{y}_3 = 0.5 * 3 + 0.5 * 4 = 3.5$$

Then, calculate the MSE loss:

$$loss = 1/3 * ((2 - 1.5)^2 + (3 - 2.5)^2 + (5 - 3.5)^2)$$

$$loss = 1/3 * ((0.5)^2 + (0.5)^2 + (1.5)^2)$$

$$loss = 1/3 * (0.25 + 0.25 + 2.25)$$

$$loss = 1/3 * 2.75$$

$$loss = 0.9167$$

16.3.2. L1 (Lasso)

The L1 regularization adds the absolute value of the magnitude of the coefficients as penalty term to the loss function. Where λ is the regularization parameter. Let's say $\lambda = 0.1$, compute the L1 regularization for the given weights.

$$L1 = loss + 0.1 * (|0.5| + |0.5|)$$

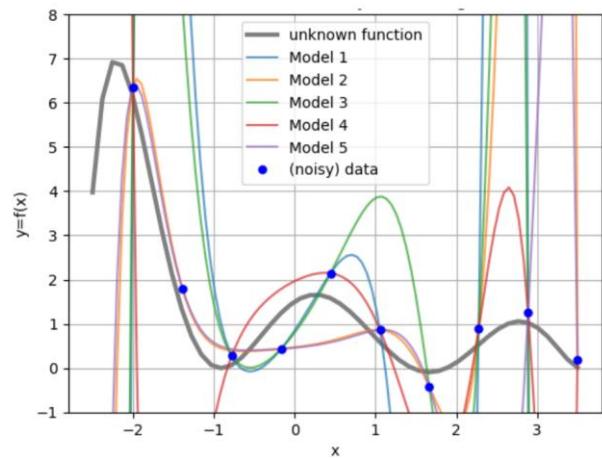
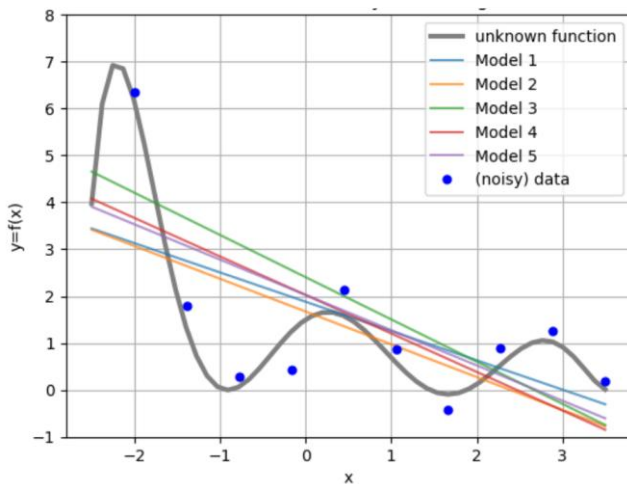
$$L1 = loss + 0.1 * (1.0)$$

16.3.3. L2 (Ridge)

The L2 regularization adds the square of the magnitude of the coefficients as penalty term to the loss function. Again, let's say $\lambda = 0.1$, compute the L2 regularization for the given weights. $L2 = loss + 0.1 * ((0.5)^2 + (0.5)^2)$

16.3.4. Polynomial Features

You are given a training dataset of 10 samples drawn from an unknown underlying function. The samples are shown in blue. Your task is to randomly select 7 data points as training data and fit a linear regression model to them. You repeat this 5 times, randomly choosing the training data. Sketch plausible plots showing the model output illustrating how the model fit the training data.



Now, instead of linear regression, use PolynomialFeatures with degree $M = 10$ to fit models of higher complexity to the same training dataset. Repeat the procedure 5 times: randomly select 7 data points as training data, fit the model, and obtain the resulting functions. Sketch plausible plots showing these fitted models.

Explain the effect of model complexity on bias and variance trade-off in the above scenario. We see that simple linear regression model has high bias and low variance. As the model complexity increases, the model has low bias and very high variance.

16.4. ÜBUNG 9

16.4.1. Standardization

Consider a 1-dimensional dataset with 4 points: $X = [1, 1, 5, 5]$. Compute the standardized values using the following two schemes:

a.) Min-Max scaling to the range $[a, b]$ where a and b forms the desired scaling range with $a=0, b=1$.

b.) Standardization to variance 1 (z-score scaling/standardization)

$$x' = \frac{x - X_{\min}}{X_{\max} - X_{\min}}$$

$$x_{\text{scaled}} = x' * (b - a) + a$$

$$\begin{aligned} X_{\min} &= 1 \\ X_{\max} &= 5 \end{aligned}$$

$$x'_1 = x'_2 = \frac{1 - 1}{5 - 1} = 0$$

$$x'_3 = x'_4 = \frac{5 - 1}{5 - 1} = 1$$

$$\mathbf{X}_{\text{scaled}} = [0, 0, 1, 1]$$

$$z = \frac{(x - \mu)}{\sigma}$$

$$\begin{aligned} \text{where } \mu &= \text{mean}(X) = \frac{1}{n} \sum_{i=1}^n x_i \\ \text{Standard deviation } \sigma &= \sqrt{\frac{1}{n} \sum (x_i - \mu)^2} \end{aligned}$$

$$n = 4$$

$$\mu = \frac{1 + 1 + 5 + 5}{4} = 3$$

$$\sigma = \sqrt{\frac{1}{4} ((1 - 3)^2 + (1 - 3)^2 + (5 - 3)^2 + (5 - 3)^2)} = \sqrt{\frac{1}{4} (4 + 4 + 4 + 4)} = 2$$

$$x'_1 = x'_2 = \frac{1 - 3}{2} = -1$$

$$x'_3 = x'_4 = \frac{5 - 3}{2} = 1$$

$$\mathbf{Z} = [-1, -1, 1, 1]$$

The regression model is: $\hat{y} = w_1 * x + w_0$

We evaluate the models using the $\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

You are given the following dataset

x	y
1	2
2	4
3	6
4	7

Use the following fold split:

Fold 1: (1,2), (2,4)

Fold 2: (3,6), (4,7)

16.4.2. Cross-Validation

We will perform 2-fold cross-validation for a simple linear regression model.

• Model 1: Trained on Fold 1 and validated on Fold 2 • Model 2: Trained on Fold 2 and validated on Fold 1. Draw the datapoints and sketch the resulting fitted linear regression models (lines). You can then read the value for $\hat{y}_i$ from the plot. Calculate the MSE for both models separately. Calculate the 2-fold cross validation error.

$$\text{CV-MSE} = \frac{\text{MSE}_1 + \text{MSE}_2}{2} = \frac{0.5 + 2.5}{2} = 1.5$$

$$\text{MSE}_1 = \frac{1}{2} ((6 - 6)^2 + (7 - 8)^2) = \frac{1}{2} (0 + 1) = 0.5 \quad \text{MSE}_2 = \frac{1}{2} ((2 - 4)^2 + (4 - 5)^2) = \frac{1}{2} (4 + 1) = 2.5$$

16.5. ÜBUNG 10

Suppose the task is to predict the admission decision of an applicant given his secondary school score (x_1) and BS grades (x_2). Given some data and using logistic regression, we train a model that has the following parameters

$$p(x) = \frac{1}{1 + e^{-(w_1 x_1 + w_2 x_2 + w_3)}}$$

1. Estimate the probability that an applicant gets accepted if Secondary school score = 4.5 and BS grade = 5.5

$$\begin{aligned} w_1 &= 1 \\ w_2 &= -0.7 \\ w_3 &= -0.7 \end{aligned}$$

2. What should be the BS grade of an applicant with secondary school score = 4.5 to have a 50 % chance of getting accepted?

$$z = (w_1 x_1 + w_2 x_2 + w_3)$$

$$p(x) = 0.5 = \frac{1}{1 + e^{-z}} \text{ substitute: } e^{-z} \rightarrow x$$

$$0.5 = \frac{1}{1 + x} \Rightarrow x = 1 = e^{-z}$$

$$e^{-z} = 1$$

$$\ln(e^{-z}) = \ln(1)$$

$$-z = 0 \Rightarrow (w_1 x_1 + w_2 x_2 + w_3) = 0$$

$$x_2 = \frac{0 - w_1 x_1 - w_3}{w_2}$$

$$x_2 = \frac{0 - 1 \cdot 4.5 - (-0.7)}{-0.7} = 5.43$$

16.6. ÜBUNG 11

16.6.1. Logistic Regression Decision Boundary

Given some data and using logistic regression, we train a model that has the following parameters

$$p(x) = \frac{1}{1 + e^{-(w_1 x_1 + w_2 x_2 + w_3)}}$$

$$\begin{aligned} w_1 &= 1 \\ w_2 &= 0 \\ w_3 &= 7 \end{aligned}$$

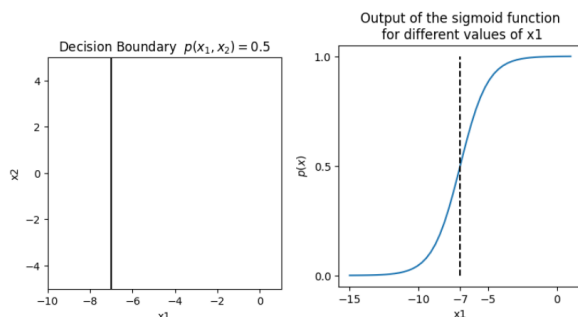
$$p(x) = 0.5 = \frac{1}{1 + e^{-z}}$$

$$0.5 = \frac{1}{1 + e^{-z}} \Rightarrow e^{-z} = 1 \Rightarrow z = 0$$

$$z = 0 \Rightarrow (w_1 x_1 + w_2 x_2 + w_3) = 0$$

$$(1 \cdot x_1 + 0 \cdot x_2 + 7) = 0 \Rightarrow x_1 = -7$$

Since $w_2 = 0$, the decision boundary is parallel to the x_2 axis at $x_1 = -7$



$$d_M(x_1, x_2) = \sum_{i=1}^p |x_{1,i} - x_{2,i}|$$

Example for $d_M(5,1)$

$$d_M(5,1) = (3 - 2) + (3 - 2) = 2$$

Manhattan distances	
$d_M(5,1)$	2
$d_M(5,2)$	2
$d_M(5,3)$	2
$d_M(5,4)$	2

Manhattan distances	
$d_M(1,2)$	4
$d_M(1,3)$	2
$d_M(1,4)$	2
$d_M(1,5)$	2

k = 3, metric = Manhattan			
Train points	Val point	true class	predicted class
1, 2, 3, 4	5	-	+
1, 2, 3, 5	4	+	+
1, 2, 4, 5	3	+	+
1, 3, 4, 5	2	+	+
2, 3, 4, 5	1	+	+

Accuracy = $(1/5) * (0 + 1 + 1 + 1 + 1) = 0.8$ i.e. 80%

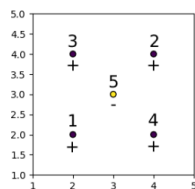
In this case, the choice of distance metric does not affect the classification with $k = 3$.

16.6.2. K-NN Classifier

We have a small dataset with 2 features. The points are (2,2), (4,4), (2,4), (4,2), and (3,3). The points (2,2), (4,4), (2,4), (4,2) belong to +ve class. The point (3,3) belongs to -ve class.

1. We now build a K-NN classifier using Euclidean distance as the distance metric. We want to evaluate the classifier with LOOCV (Leave One Out Cross Validation). Calculate the accuracy for LOOCV for $k=1$, and $k = 3$.

2. Suppose we change the distance metric to Manhattan distance. Do you see any effect on classification with $k=3$?



Euclidean distances	
$d_E(5,1)$	$\sqrt{2}$
$d_E(5,2)$	$\sqrt{2}$
$d_E(5,3)$	$\sqrt{2}$
$d_E(5,4)$	$\sqrt{2}$

Euclidean distances	
$d_E(1,2)$	$\sqrt{8}$
$d_E(1,3)$	2
$d_E(1,4)$	2
$d_E(1,5)$	$\sqrt{2}$

k = 1, metric = Euclidean			
Train points	Val point	true class	predicted class
1, 2, 3, 4	5	-	+
1, 2, 3, 5	4	+	-
1, 2, 4, 5	3	+	-
1, 3, 4, 5	2	+	-
2, 3, 4, 5	1	+	-

k = 3, metric = Euclidean			
Train points	Val point	true class	predicted class
1, 2, 3, 4	5	-	+
1, 2, 3, 5	4	+	+
1, 2, 4, 5	3	+	+
1, 3, 4, 5	2	+	+
2, 3, 4, 5	1	+	+

Accuracy = $4 / 5 = 0.8$ i.e., 80%

$$d_E(x_1, x_2) = \sqrt{\sum_{i=1, p} (x_{1,i} - x_{2,i})^2}$$

Example for $d_E(5,1)$

$$d_E(5,1) = \sqrt{(3 - 2)^2 + (3 - 2)^2} = \sqrt{2}$$

16.7. ÜBUNG 13: ENSEMBLE

16.7.1. Hard voting with weights

An AI-assisted Email classification system uses Ensemble of three classifiers: 1. Logistic regression `log_clf`, 2. 3-Nearest neighbour `knn_clf3` and 3. 10-Nearest neighbour to predict class spam (class =1), ham (class =0). The Ensemble uses hard voting with weights. The weights are defined as [0.1, 0.3, 0.6] respectively. For a particular email, the predictions of the three classifiers are [ham, ham, spam], respectively. What is the final class predicted by the Ensemble?

Hard voting is a method of combining the predictions of multiple classifiers into a single prediction. In hard voting, the predictions of each classifier are summed for each class label, and the class label with the most votes is chosen as the final prediction. When using weights in hard voting, each vote is weighted according to the specified weight. For this example, we have: *The weighted score for ham* = $0.1 + 0.3$ *The weighted score for spam* = 0.6 So we get: *ham*: 0.4 , *spam*: 0.6 Since the weight of the third classifier, which is predicting spam, is greater than the combined weights of the other two classifiers, the final prediction of the Ensemble would be spam.

16.7.2. Ensemble Soft Voting

For spam-ham classification problem, we have three classifiers (`clf1`, `clf2`, `clf3`) and the ensemble uses soft voting. For an Email, the classifiers return the following probabilities (`predict_proba`): $Pr_clf1(Spam) = 0.6$, $Pr_clf2(Spam) = 0.8$, $Pr_clf3(Spam) = 0.2$ Will the Ensemble classify the Email as Spam or Ham?

In this case, the three classifiers have predicted the following probabilities for the spam class: [0.6, 0.8, 0.2]. When these probabilities are averaged, the resulting average for spam is $(0.6 + 0.8 + 0.2) / 3 = 0.53$. Since this average is greater than 0.5, the Ensemble would classify the email as spam.

16.7.3. Ensemble Soft Voting with weights

For spam-ham classification problem, we have two classifiers (`clf1`, `clf2`) and we use an ensemble method soft voting with weighted averages. The weights for the classifiers are $w_clf1 = 1.2$, $w_clf2 = 0.8$ For an Email, $Pr_clf1(Spam) = 0.8$, $Pr_clf2(Spam) = 0.4$ Will the ensemble classify the Email as Spam?

In this case, the two classifiers have predicted the following probabilities for the spam class: [0.8, 0.4], and the corresponding weights are [1.2, 0.8]. When these probabilities are multiplied by their corresponding weights and then averaged, the resulting weighted average is $(0.8 * 1.2 + 0.4 * 0.8) / 2 = 0.64$. Since this weighted average is greater than 0.5, the Ensemble would classify the email as spam.

16.7.4. Ensemble Soft Voting with weights

We are training AdaBoost on a dataset of size 100. A weak learner that we have trained classifies 68 out of 100 samples correctly. What is the predictor weight that AdaBoost will assign to this predictor?

First, we assign a sample weight to each sample. *Sample weight* = $1 / \# \text{ of samples} = 0.01$ In this case, the predictor has classified 68 out of 100 samples correctly. We now need the accuracy of this classifier. *accuracy* = $68 * 0.01 = 0.68$ The predictor weight is then calculated by the following formula. *predictor weight* = $\ln(1 - \text{accuracy})$ So, for this instance we get *predictor weight* = $\ln(1 - 0.68)$, *predictor weight* = 0.754

16.8. ÜBUNG 14

16.8.1. Softmax

A deep neural network classifies images of animals as one of these four classes: cat, dog, bird, or fish.

For a given image, the network outputs the following scores-vector $\vec{s}_k = [1, 6, 2, 3]$.

We can apply the softmax function to this vector to transform these scores into probabilities.

Apply the softmax function given by this formula and calculate the probabilities.

- $p_k = \frac{\exp(s_k(x))}{\sum_{j=1}^M \exp(s_j(x))}$
- M is the total number of classes
- $s_k(x)$ is the score for each class k

$$\begin{aligned}\exp(s_1(x)) &= e^1 = 2.718 \\ \exp(s_2(x)) &= e^6 = 403.429 \\ \exp(s_3(x)) &= e^2 = 7.389 \\ \exp(s_4(x)) &= e^3 = 20.086 \\ p_1 &= 2.718/433.622 = 0.006 \\ p_2 &= 403.429/433.622 = 0.93 \\ p_3 &= 7.389/433.622 = 0.017 \\ p_4 &= 20.086/433.622 = 0.046\end{aligned}$$

Calculate the exponential of each score:

$$[0.006, 0.93, 0.017, 0.046]$$

Calculate the sum of all the exponentials: $\sum_{j=1}^{M=4} \exp(s_j(x)) = 2.718 + 403.429 + 7.389 + 20.086 = 433.622$

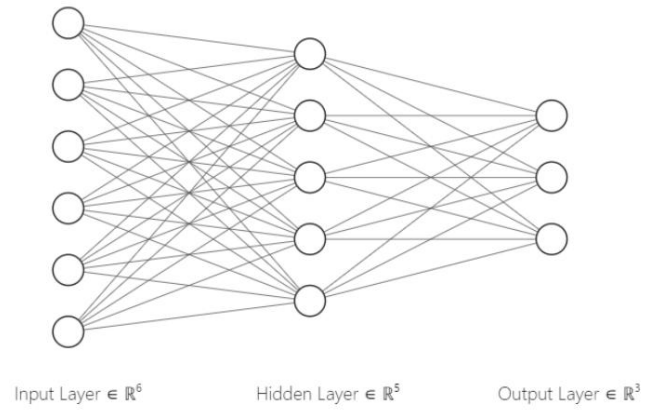
Divide each exponential by the sum of all exponentials to get the final output:

The output of the softmax function can be interpreted as a probability distribution over the set of scores that were input to the function. The values in the output array sum to 1, and each value represents the probability that the image belongs to that class.

16.8.2. Number of parameters

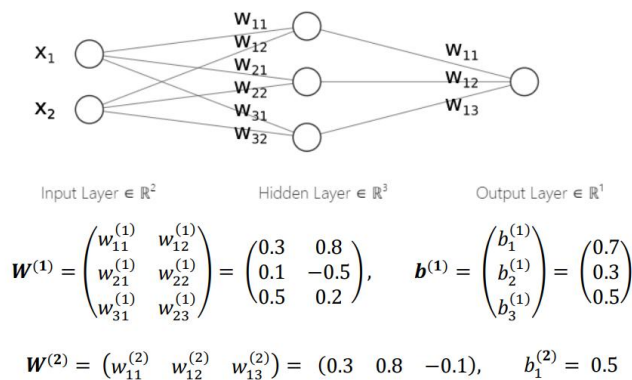
You train a neural network with an input layer with 6 units, a hidden layer with 5 units and an output layer with 3 units. Calculate how many trainable parameters this neural network has.

The input layer just represents the input vector of length 6 and has no trainable parameters. The hidden layer has for each unit a weight for every unit in the input layer plus a bias parameter. $number_of_parameters_hidden = (6 * 5) + 5 = 35$ The output layer has for each unit a weight for every unit in the hidden layer plus a bias parameter $number_of_parameters_output = (5 * 3) + 3 = 18$ So the total number of trainable parameters is: $number_of_parameters_total = 35 + 18 = 53$



16.8.3. Forward Pass

Calculate the output of this given neural network, when inputting $\mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$



$$ReLU(x) = \begin{cases} x & \text{if } x > 0, \\ 0 & \text{otherwise} \end{cases}$$

Forward pass is given by
 pre-activation hidden layer: $\mathbf{z}^{(1)} = \mathbf{W}^{(1)} * \mathbf{x} + \mathbf{b}^{(1)}$
 activation hidden layer: $\mathbf{h}^{(1)} = ReLu(\mathbf{z}^{(1)})$
 pre-activation output layer: $\mathbf{z}^{(2)} = \mathbf{W}^{(2)} * \mathbf{h}^{(1)} + \mathbf{b}^{(2)}$
 model output: $\hat{\mathbf{y}} = ReLu(\mathbf{z}^{(2)})$

SOLUTION:

$$\mathbf{z}^{(1)} = \begin{pmatrix} 0.3 & 0.8 \\ 0.1 & -0.5 \\ 0.5 & 0.2 \end{pmatrix} * \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 0.7 \\ 0.3 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 2.6 \\ -0.6 \\ 1.4 \end{pmatrix}$$

$$\mathbf{h}^{(1)} = ReLu\left(\begin{pmatrix} 2.6 \\ -0.6 \\ 1.4 \end{pmatrix}\right) = \begin{pmatrix} 2.6 \\ 0 \\ 1.4 \end{pmatrix}$$

$$\mathbf{z}^{(2)} = (0.3 \quad 0.8 \quad -0.1) * \begin{pmatrix} 2.6 \\ 0 \\ 1.4 \end{pmatrix} + 0.5 = 1.14$$

$$\hat{\mathbf{y}} = ReLu(1.14) = \mathbf{1.14}$$

Instead of the matrix calculations, one can also solve the equations for each neuron one by one.

This is probably easier to understand but not computationally efficient.

Output of neuron 1 in the hidden layer 1 is

$$\begin{aligned} h_1 &= Relu(w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}) \\ h_2 &= Relu(w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)}) \\ h_3 &= Relu(w_{31}^{(1)} x_1 + w_{32}^{(1)} x_2 + b_3^{(1)}) \end{aligned}$$

Final Output

$$\hat{\mathbf{y}} = ReLu(w_{11}^{(2)} h_1 + w_{12}^{(2)} h_2 + w_{13}^{(2)} h_3 + b_1^{(2)}) = \mathbf{1.14}$$